

QA_DISCOVERY_LEDGER

QA Discovery-Channel Ledger

Revision: 17 **Last modified:** 2026-08-25T19:55:00Z **Status:** active

Constitution: §11.4.238 (automated QA must be the DISCOVERER, not the confirmer — every defect found outside the automated HelixQA regime is itself a coverage-escape release blocker, not merely a bug to fix).

Purpose

Every defect this project closes **MUST** record **which channel found it**: automated-helixqa (the standing HelixQA regime caught it) or an **out-of-band channel** — manual-qa, operator-report, agent-code-reading, or incidental-discovery. Per §11.4.238(C), every out-of-band entry **MUST** carry a **coverage-escape audit**: why the automated regime missed it (cited to the specific missing/blind check, never “we didn’t think of it”), and the new or strengthened automated check — with its own §11.4.115 RED capturing the escaped defect — that closes the gap.

Per §11.4.238(E), this project tracks the discovery-channel split over time and drives the out-of-band share toward zero. **Honest boundary (§11.4.6)**: this ledger starts wherever this project’s actual history is — see the retroactive seed entries below, all found via agent-code-reading during the 2026-08-07/2026-08-08 governance audits, exactly the channel this anchor targets. Full retroactive audit of every historical closure in docs/Fixed.md (~62 items) is NOT attempted here — that is out of proportion for a ledger just being stood up, and would itself be a §11.4.6 fabrication if backfilled without real investigation per entry. The ledger is honest about starting now, not claiming a false complete history.

Schema (per entry)

Field	Meaning
id	The workable item id (BOB-NNN) if one exists, else a short slug
date	ISO date the defect was found
channel	automated-helixqa manual-qa operator-report agent-code-reading incidental-discovery
summary	One line: what was wrong
escape-audit	(out-of-band only) which check was missing/blind, cited by name/path
new-check	(out-of-band only) the new/strengthened check that now catches this class, with its RED-capture evidence

Entries

RD2-22 — PATCH /schedules/{id} and PUT /theme unauthenticated with a token set

- **id:** RD2-22 (governance-audit item, not yet a BOB-NNN)
- **date:** 2026-08-07 (found by the 2026-08-07 governance audit), confirmed still open + fixed 2026-08-08
- **channel:** agent-code-reading — found by direct source inspection (comparing sibling routes' Depends(require_api_token) presence), not by any test or HelixQA run.
- **escape-audit:** tests/security/test_hooks_schedules_auth.py's _MUTATING enumeration (the ONLY automated check covering this auth surface) was scoped to exactly 4 routes (POST/DELETE hooks, POST/DELETE schedules) when the original RW-02 fix landed — it was never extended when PATCH /schedules/{id} and PUT /theme were added as separate routes. The check existed, ran green, and was simply never told about two of the six routes it should have covered — a scope gap, not a broken check (per the ledger's channel split, this is the “genuinely uncovered” class, not “existed-but-missed”).

- **new-check:** `_MUTATING` extended to all 6 routes (`tests/security/test_hooks_schedules_auth.py`), RED-captured (confirmed 4/4 new-route assertions failed with expected 401, got 200 before the fix), now GREEN (31/31). See `docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md` RD2-22 for the full before/after evidence.

RD2-41a — `scripts/docs_chain.sh` Step 1/3 silently no-op'd on every run

- **id:** RD2-41 (governance-audit item)
- **date:** 2026-08-08
- **channel:** `agent-code-reading` — found by directly invoking `docs_chain.sh` and reading its own printed error line, then tracing the hardcoded path in source.
- **escape-audit: no automated check existed at all** for “does `docs_chain.sh`’s Step 1 actually run the real DB export” — the script printed an `ERROR: line` on every failure but nothing consumed/ asserted on that exit status anywhere in the test suite or pre-build gate; a human had to actually run it and read the output. Genuinely uncovered, not a broken check.
- **new-check:** `tests/unit/test_docs_chain_binary_resolution.sh` — real-invocation test asserting Step 1 resolves a real binary and reaches the real `validate` call. RED-captured (confirmed failing against the pre-fix script: `FAIL: Step 1/3 still reports 'binary not found'`), now GREEN.

RD2-41b — `scripts/pre_build_verification.sh` invariant 17 silently SKIPPED on every run

- **id:** RD2-41 (governance-audit item, same root cause as RD2-41a)
- **date:** 2026-08-08
- **channel:** `agent-code-reading` — found while fixing RD2-41a, by checking for the identical bug pattern in this sibling script per the project’s own extend-to-all-cases discipline.
- **escape-audit:** this is the sharpest instance in this ledger of “the checker itself was blind”: invariant 17 IS the pre-build gate’s own workable-items DB-integrity check, and it was silently SKIPPING (not failing — the script’s own `else` branch treats a missing binary as an honest skip, not a failure) on every single pre-build run since the path was wrong. A gate whose own precondition-check is broken cannot report the truth about the thing it gates — an `existed-but-missed` class defect (the check existed, ran, and was

blind), the more dangerous of the two classes per this ledger's schema.

- **new-check:** `tests/unit/test_pre_build_workable_items_invariant.sh` — real-invocation test asserting invariant 17 reaches a real PASS/FAIL verdict instead of silently skipping. RED-captured (confirmed failing against the prefix script via `git stash/restore`), now GREEN. **Residual, still-open finding from this same investigation** (tracked as RD2-41's own note in `docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md`, not yet closed): a SEPARATE, earlier mutation-marker carrier false-positive in the same script aborts the ENTIRE pre-build gate before invariant 17 is ever reached in a full run — meaning this fix, while correct, is not yet provably exercised end-to-end in production; the ledger records this honestly rather than claiming full closure.

BOB-008 — DB↔MD body drift + missing enumerated unblock choices

- **id:** BOB-008
- **date:** 2026-08-08
- **channel:** agent-code-reading — found by running `workable-items diff/validate` during a routine sync check, not by any standing automated gate (no CI job runs `workable-items diff` on a schedule).
- **escape-audit:** no automated check runs `workable-items diff/validate` as a standing, scheduled, or write-seam-triggered gate — §11.4.106(F)'s commit-seam sync hook (which would have caught the DB write in commit 54e313f landing with no matching `docs/Issues.md` update) does not exist in this repo. Genuinely uncovered.
- **new-check: closed 2026-08-08.** `scripts/pre_build_verification.sh` invariant 17 extended to run `workable-items diff` (DB-vs-Markdown divergence) alongside `validate` (internal DB invariants) — it now catches BOTH the §11.4.148(D3) enumerated-choices class AND the DB↔MD body-drift class. RED-captured via `tests/unit/test_pre_build_workable_items_diff_check.sh`'s §1.1 paired mutation (a deliberately desynced `docs/Issues.md` — confirmed FAIL with the diff check absent, confirmed correct divergence-specific FAIL, distinguished from the separate pre-existing BOB-009/010 `validate` issue so the test does not tautologically reuse an unrelated failure — then GREEN after the fix, with a companion assertion that the real synced tree reports no false divergence).
- **2026-08-18 update (BOB-073 / RD2-04, recurrence, honest residual finding):** despite the 2026-08-08 invariant-17 diff extension above, `workable-items diff --db docs/workable_items.db --issues docs/Issues.md --fixed docs/Fixed.md`

found **9 separate DB↔Markdown drifts** (not just the original BOB-008 one) when an agent manually re-ran it during the BOB-072/BOB-073 fix session (commit 82d9842) — evidence in docs/qa/BOB-072-073/. **Channel for this recurrence: agent-code-reading** (an agent ran the tool by hand while fixing an unrelated sibling item, not a standing scheduled/blocking gate). **Escape-audit:** the invariant-17 diff check landing in source is necessary but not proven sufficient — this ledger does NOT have evidence that invariant 17 runs as a hard, blocking write-seam gate on every commit that touches docs/workable_items.db (per §11.4.106(F)’s commit-seam requirement); 9 drifts accumulating between 2026-08-08 and 2026-08-18 is consistent with either (a) the check existing but only being invoked when an agent remembers to run scripts/pre_build_verification.sh by hand, or (b) some DB writes in that window landing via a path invariant 17 does not intercept. **Recorded honestly as UNKNOWN which mechanism** (§11.4.6) rather than guessed. All 9 reconciled + closed this session (see commit 82d9842 for the per-item DB-vs-MD authoritative-side decisions); diff now reports 0. **New check (still owed, not yet authored):** a real commit-time (pre-commit or commit-push-all.sh stage per §11.4.234) hook that runs workable-items diff and BLOCKS the commit on any divergence touching docs/workable_items.db, docs/Issues.md, or docs/Fixed.md — turning invariant 17 from a pre-build-time report into a write-seam gate.

RD2-42 — podman ps reports a container “Up ... (healthy)” while its process is genuinely dead

- **id:** RD2-42 (governance-audit item)
- **date:** 2026-08-09
- **channel:** incidental-discovery — surfaced while attempting the RD2-22 live curl-verify: curl http://localhost:7187/health returned HTTP=000 (connection refused), while podman ps simultaneously reported qbittorrent-proxy: Up 15 hours (healthy). podman exec qbittorrent-proxy ... returned OCI runtime error: crun: ... is not running and podman inspectNetworkSettings.Ports was empty {} — the container’s actual crun process was dead (no restart count, no OOMKilled flag recorded — podman’s own state was simply stale/desynced from reality), most likely fallout from the same host session-kill mechanism already tracked in docs/incidents/ reaching into the rootless-podman container process tree.
- **escape-audit: no automated check exists at all** for “is podman ps’s reported state consistent with the container’s real crun/process

state” — this is exactly the §11.4.196(F)/§11.4.201(6) “configured $\neq$ in use” false-null class applied to container orchestration: a healthy-looking `podman ps` line is not proof the service is reachable.

Genuinely uncovered — no pre-build/pre-test gate probes the live stack’s actual reachability before a test run assumes it.

- **new-check:** not yet authored (tracked, not yet closed). Candidate: a `tests/fixtures/services.py`-level or pre-build-gate-level real-reachability probe (`curl/socket-connect` to each mapped port, not `podman ps` text) run before any live-stack- dependent test session, with an honest recreate-and-retry path — mirroring what this session did manually (`./start.sh --recreate`, confirmed `HTTP=200` after ~90s).
- **resolution applied this session:** `./start.sh --recreate` (the project’s sanctioned orchestrator, never `raw podman restart`) — confirmed `curl :7187/health → 200` after the stack’s normal ~90s startup window.

RD2-43 — bare python3 -m pytest silently fails ALL collection via a stale ~/.local rpds build

- **id:** RD2-43 (governance-audit item)
- **date:** 2026-08-09
- **channel:** incidental-discovery — surfaced when relaunching the interrupted `tests/contract/`
 - `tests/unit/regression` sweep with a `barepython3 -m pytest` invocation: `immediateModuleNotFoundError: No module named 'rpds.rpds'` at collection time (inside `theschemathesis` pytest-plugin’s import chain`), aborting the ENTIRE sweep before a single test ran.
- **escape-audit:** host-level Python version drift — the system `python3` resolves to Python 3.14.6, but `~/.local/lib/python3/site-packages/rpds/rpds.cpython-313-x86_64-linux-gnu.so` is a native extension built for the 3.13 ABI (real ABI mismatch, confirmed via `ls .../rpds/*.so + python3 --version`). The project’s own `.venv` (Python 3.14.6, `.venv/bin/ python3 -c "import rpds"` succeeds) is the correct, working interpreter — no automated check enforces “tests are always run via `.venv/bin/python3`, never a bare `python3`” anywhere in this repo’s own tooling (no `Makefile`/wrapper script that fails closed on the wrong interpreter). Genuinely uncovered; this is an environment-fragility class distinct from the source-level bugs this ledger otherwise tracks, but it silently produces a 100%-collection-failure that could be misread as “every test in the suite is broken” by anyone (human or agent) who doesn’t already know to check the interpreter.

- **new-check:** not yet authored (tracked, not yet closed). Candidate: a thin `scripts/run_tests.sh` wrapper (or a `pyproject.toml`/CI-adjacent guard) that verifies `sys.prefix` resolves inside `.venv/` before invoking `pytest`, failing closed with an actionable message rather than a confusing plugin-internals traceback.
- **resolution applied this session:** relaunched the sweep via `.venv/bin/python3 -m pytest` — confirmed genuinely running (real per-test PASS lines, not a collection error).

RD2-44 — `test_get_existing_search_returns_200`'s 120s poll window had near-zero real margin

- **id:** RD2-44 (governance-audit item, same root-cause FAMILY as the already-fixed `tests/e2e/test_full_pipeline.py` timeouts, but a genuinely separate finding/fix — that fix did not touch this file)
- **date:** 2026-08-09
- **channel:** incidental-discovery — surfaced re-running `tests/integration/test_merge_api.py` (RD2-26a's own deliverable) against the actually-live stack for the first time since its original mocked-service replacement; it had only ever been run with the stack unreachable (28 passed / 20 skipped) until this session.
- **escape-audit:** RD2-26a's own author (a parallel subagent, same session) had no way to discover this — the file was authored and its author's own live-verification attempt happened while the stack was down, so it never actually exercised this code path against a live, contended host before committing. No automated check re-runs the live-service test suite on a schedule independent of whether an agent happens to have the stack up at authoring time — genuinely uncovered.
- **new-check:** the test itself IS the check; it was simply mistimed. Live-measured real search completion (42 trackers, real network calls, under real concurrent 4-subagent host load): ~118s once, ~298s under heavier concurrent load — both within a few seconds of, or past, the old 120s ceiling. Widened to 300s in `tests/integration/test_merge_api.py`, matching the `test_full_pipeline.py` pattern's order of magnitude. RED-adjacent evidence: the original 120s-deadline run genuinely failed (`assert 'running' != 'running'`) against the live stack; post-fix re-run: 1 passed in 297.72s.

INC-2026-08-10 — GCM auth-bypass mutation left in qBitTorrent-go/internal/db/crypto.go during Agent H's RED window

- **id:** INC-2026-08-10 (this-session incident; not a governance-audit item — inline coverage-escape captured during autonomous work by the conductor)
- **date:** 2026-08-10
- **channel:** automated_background_scan — a background security-review plugin (out-of-cycle from the pre-build regime) caught the mutation during Agent H's Case (c) RED-then-GREEN window (~few seconds) while it was authoring RD2-31 chaos tests in qBitTorrent-go/tests/integration/jackett_db_test.go.
- **escape-audit:** subagent briefing template for RD2-31 said “Do NOT touch production code paths — ONLY the test file” but was PROSE, not mechanically enforced. Agent H interpreted narrowly and mutated qBitTorrent-go/internal/db/crypto.go Decrypt (if err != nil → if false && err != nil, swallowing GCM authentication failures) for its RED assertion, then restored to byte-identical HEAD. The window was small but real — a §11.4.84 working-tree-quiescence violation. No boba-side pre-tool-use hook would have caught the mutation during the window (only a post-fact scan would); the out-of-cycle security scan caught it BEFORE restoration. Genuinely uncovered by the in-cycle regime.
- **new-check:** §11.4.84 fence LANDED —
 - **Layer 1** (subagent briefing template with three mandatory isolation patterns: git worktree per §11.4.179, atomic mutate-run-restore Bash tool call with trap, mutation-testing tool operating on isolated copy) APPLIED to every subsequent subagent dispatch (proven: Agents K + L both used the strengthened briefing without violation).
 - **Layer 3** (scripts/pre_build_verification.sh invariant 23 CM-NO-PRODUCTION-MUTATION-RESIDUE, with FIXTURE_ROOT env-testable golden-good/golden-bad fixtures under scratchpad/agent-L-fixtures/) LANDED — real-repo PASS today (0 residue in production paths).
 - **Layer 2** (post-tool-use hook enforcing mid-window detection) DEFERRED pending Claude Code runtime-capability verification (currently PreToolUse hooks only per §11.4.109 precedent).
 - Design doc: scratchpad/task-20-84-fence-design.md.
- **resolution applied this session:** Agent H's crypto.go restoration was clean (verified via git status empty + git diff empty + full-tree grep for MUT'ATED patterns in production paths = empty); no commit contamination (main stream was holding for atomic

commit per §11.4.121). §11.4.84 fence Layer 1 + Layer 3 both landed and verified. Layer 2 tracked as §11.4.197 follow-up.

TMUX-OOMD-2026-08-12 — tmux sessions SIGKILLd by systemd-oomd under user-slice pressure, despite MemoryMax=infinity

- **id:** TMUX-OOMD-2026-08-12 (not yet a BOB-NNN — cross-project escape traced through the upstream vasic-digital/tmux fix TMX-083, v1.0.42)
- **date:** 2026-08-12
- **channel:** operator-report — the operator noticed sessions dying “as soon as we continue work with this project”. No automated gate in this project or its upstream had exercised this failure class.
- **escape-audit:** neither the tmx project’s own test suite (which covers cgroup Max= / TasksMax= / CPUQuota properties but not ManagedOOMPreference) nor the boba project’s HelixQA + Challenges regime (host-safety mandates cover CONST-033 host power classes but do not probe systemd-oomd’s effect on tmux scopes) had a check for “does the tmux scope survive a user-slice memory-pressure spike under systemd-oomd?”. The §11.4.201(6) FALSE-NULL class applied to the test surface: the suite scanned the wrong axis and returned a confident zero on a real defect. Genuinely uncovered — no automated check existed on either side. Predecessor sighting RD2-42 (2026-08-09, incidental-discovery) had already noted the same host session-kill mechanism reaching into rootless-podman container process trees; this operator report is the tmux-facing symptom of the same killer.
- **new-check:** challenges/scripts/tmux_survives_oomd_pressure_challenge.sh (boba side; auto-wired into run_all_challenges.sh)
 - upstream scripts/tests/59_oomd_preference_avoid.sh (tmx side). Both are §11.4.115 RED-capable via RED_MODE=1. Live-verified on the operator’s host 2026-08-12: pre-fix RED_MODE=1 PASS (ManagedOOMPreference=none — defect reproduced), post-fix RED_MODE=0 PASS (ManagedOOMPreference=avoid — regression guard confirmed). Perfect §11.4.115 polarity flip. Full audit at docs/qa/coverage-escape-tmux-oomd-20260812/audit.md. Root cause fixed upstream in vasic-digital/tmux v1.0.42 (TMX-083, commits 6f9eaeb + merge 92ef3a0), pushed ff-only to both remotes per §11.4.113.

RD2-00 / BOB-068 — unattributed, unreviewed “Auto-commit” mechanism pushing to main mid-work

- **id:** BOB-068 (RD2-00) — status Queued, still OPEN (root cause narrowed, not eliminated)
- **date:** first observed 2026-08-08; re-confirmed present in git history as of 2026-08-18 (20 bare Auto-commit commits total across this repo’s history — confirmed via `git log --oneline --all --grep="^Auto-commit$" | wc -l`, e.g. 54e313f, 9c8f684, 743097a, de9270b, 1c36777, 41179c2, 7c529ca).
- **channel:** agent-code-reading — found by re-running `git log` mid-investigation during the 2026-08-08 governance audit and discovering two NEW Auto-commit commits (9c8f684, 743097a) had landed on `main` **while the audit was already in progress**.
- **escape-audit:** no automated check exists anywhere in this repo for “does a commit reaching `main` carry an ATM-NNN citation / TDD trail, or is it a bare/templated message from an unattributed source” — §11.4.84 working-tree-quiescence has no mechanical guard on this specific path (writes arriving via an ordinary `git pull --ff` from a second session/host with push access to the same remotes, per the RD2-00 root-cause pass in docs/`GOVERNANCE_AUDIT_2026-08-08_ROUND2.md`: `git reflog` proved the two newest commits were never authored on the investigating host — they fast-forwarded in via `pull`, and their commit timezone (+0500) does not match the investigating host’s own (+0300 MSK)). Genuinely uncovered — no gate flags an unattributed/unticketed commit reaching `main`.
- **new-check:** not yet authored (tracked, not yet closed — BOB-068 remains Queued). Followup filed: **BOB-106** — a §11.4.84 quiescence-check helper scanning the commit range since the last known-good release tag and FAILing on any bare/templated commit message (^Auto-commit\$, ^sync:, etc.) with no ATM-NNN reference, wired into `scripts/pre_build_verification.sh` or the §11.4.234 `commit-push-all.sh` entrypoint.
- **resolution status:** root cause NARROWED, not eliminated — docs/`GOVERNANCE_AUDIT_2026-08-08_ROUND2.md` RD2-00 Update 2 confirms the mechanism is (most likely) the operator’s own second session/device bypassing per-commit TDD/review discipline on ITS side, not an unidentified external actor; this downgrades severity from P0 to P1 but does not close the item.

BOB-072 — docs/workable_items.db machine-caught SSoT integrity violations (90% of closures had zero audit trail)

- **id:** BOB-072 (RD2-03) — **closed 2026-08-18**, commit 82d9842
- **date:** found 2026-08-08 (governance audit); closed 2026-08-18
- **channel:** agent-code-reading at discovery (2026-08-08, an agent manually ran constitution/scripts/workable-items/bin/workable-items validate --db docs/workable_items.db during the audit — exit 1, 2 real violations: BOB-009/BOB-010 both had a closure evidence_path that does not resolve). Beyond the tool's own catch, a direct SQL sweep at the time found 56 of 62 closed items (90%) with zero item_history rows.
- **escape-audit:** at the time of the 2026-08-08 discovery, invariant 17 of scripts/pre_build_verification.sh — the ONLY standing gate wired to run workable-items validate — was ITSELF silently SKIPPING on every run (the sibling coverage escape already ledgered above as **RD2-41b**, same date, same audit): a gate whose own precondition-check is broken cannot report the truth about the thing it gates. RD2-03's violations therefore went unnoticed by the standing regime until RD2-41b's own fix (same session) restored invariant 17 to a real PASS/FAIL verdict — the existed-but-missed class (§11.4.201(6) FALSE-NULL), not genuinely-uncovered.
- **new-check:** challenges/scripts/workable_items_integrity_challenge.sh extended with §11.4.115 RED_MODE polarity (RED_MODE=1 reproduces on the pre-fix DB snapshot in docs/qa/BOB-072-073/, RED_MODE=0 default GREEN guard — both validate and diff exit 0), 3/3 PASS live-verified this session. Combined with the already-landed RD2-41b fix, invariant 17 is now the mechanized regime for this defect class.
- **Note (discovery-method signal, not a channel reclassification):** this session's closing verification (commit 82d9842) captured its before/after evidence via the ACTUAL tool (workable-items validate/diff exit codes), not raw SQL or narrative inspection — the correct, mechanizable shape §11.4.238 wants going forward, even though the ORIGINAL 2026-08-08 discovery was still an agent running that tool by hand during an ad-hoc audit, not a standing/scheduled gate firing on its own. The channel is honestly recorded as agent-code-reading, not automated-helixqa — the tool was correct, its invocation was not yet automatic.

META-11.4.227B-2026-08-18 — §11.4.227(B) anchor-block-integrity verified BY HAND, not by a mechanical gate

- **id:** none yet (governance meta-finding, not a BOB-NNN — the underlying anchor corpus is constitution-submodule-owned; this ledger records the escape because it was found during boba-repo work this session)
- **date:** 2026-08-18 (this session's §11.4.140/§11.4.141 anchor-number-collision resolution, Phase 0 — boba commit 136a22c, constitution commits 5ed8c80..e5f2891)
- **channel:** agent-code-reading — the Phase 0 subagent manually computed md5 hashes of the moved anchor bodies and manually grepped `### §11.4.140 / ### §11.4.141` occurrence counts, pasting the results verbatim into the commit message (`git show --stat 136a22c`) to prove §11.4.227(B) block-integrity (exactly-once per anchor, byte-identical lockstep across mirrors).
- **escape-audit:** §11.4.227(B) is fully specified in constitution prose (“propagation gates count BLOCK-STARTS never bare literals: exactly-once per anchor per file... lockstep content-hash equality across the mirror set... anchor-number collisions FAIL”) but its own gate (CM-ANCHOR-BLOCK-INTEGRITY) is explicitly documented, by the anchor's own text, as “gate-code = separate work item, NOT claimed shipped.” The §11.4.140/§11.4.141 collision that §11.4.227 itself cites as its founding forensic example was STILL only caught/fixed by a human-in-the-loop agent running ad-hoc `md5sum/grep` commands, not by a runnable script — the textbook §11.4.227 finding (413 named CM-* gates, 58% unimplemented) recurring live, in this repo, this session.
- **new-check:** OUT OF SCOPE for this ticket (constitution-submodule-owned mechanism; per this task's own constraints, boba may not touch anything constitution-related). Followup filed: **BOB-105** — a boba-side, read-only challenge that greps every governance file this project's constitution submodule checkout exposes for `### §11.4.NNN` heading occurrences and FAILs on >1-per-anchor-per-file or on two DIFFERENT anchor bodies sharing one NNN, implementing (from the consumer side, read-only) the check CM-ANCHOR-BLOCK-INTEGRITY names but does not yet run.

SCRATCH-LOSS-2026-08-18 — Phase 1a subagent's declared source-material inputs absent at dispatch time

- **id:** none yet (this-session incident, constitution-curriculum work stream, not a BOB-NNN)
- **date:** 2026-08-18

- **channel:** agent-code-reading — the Phase 1a subagent (a1cc331d) discovered, at task start, that 5 source files its own task brief named as required reads (curriculum_amendment_plan_v1.md, ai_curriculum_modules_27_35_extracted.md, and three curriculum_analysis_modules_*.md gap-analysis files) were NOT present in the session scratchpad. Self-documented in .superpowers/sdd/task-phase1a-report.md:21. Independently re-verified during this BOB-069 audit: curriculum_amendment_plan_v1.md is STILL absent from the live scratchpad directory as of this writing, while the other four listed inputs now exist (apparently re-created by a later pass).
- **escape-audit:** root cause per the Phase 1a subagent’s own investigation: “the prior stub-expansion subagent (ae59171f) apparently hit its session rate limit before writing them” — a §11.4.147(e)-class API-quota-exhaustion crash landing mid-deliverable, with no mechanical dependency check verifying a declared upstream artifact actually exists before a downstream subagent is dispatched to consume it. No automated check exists in this project’s orchestration tooling that a task brief’s named “read this first” inputs are present at dispatch time — genuinely uncovered; discovered only because the downstream subagent noticed and self-reported rather than silently fabricating content against an absent source (which its own brief’s no-fabrication instruction — [MATERIAL-THIN] marking — correctly steered it away from).
- **new-check:** not yet authored. Followup filed: **BOB-107** — a pre-dispatch precondition check in the orchestration layer verifying every file a task brief cites as a required input exists and is non-empty before the downstream agent is spawned, failing closed with an actionable “missing input, respawn the producer” message.
- **Honest boundary (§11.4.6), stated explicitly:** the task brief that dispatched this BOB-069 audit described this finding as “scratchpad reset between sessions” — that specific mechanism (an OS-level / tmp wipe or session-boundary reset) is NOT what the cited evidence actually shows; the evidence instead shows a crashed/rate-limited PRODUCER subagent that never wrote its declared deliverable, discovered by an honest downstream subagent. This entry records the mechanism the evidence actually supports, not the dispatching brief’s hypothesis, per this ledger’s own anti-fabrication discipline (§11.4.6/§11.4.238’s “no fabricated entries” instruction).

CODEGRAPH-1.5.0-GITIGNORE-2026-08-18 — CodeGraph 1.5.0 re-index walked into nested-.gitignore-excluded node_modules trees (63x file-count blowup)

- **id:** none yet (this-session incident; followup tracked as BOB-104)
- **date:** 2026-08-18 — discovered by BOB-075 subagent a70f216f (per `.superpowers/sdd/progress.md:78`, dispatched as Task #47) while attempting a mechanical docs/codegraph/Status.md regen; landed in commit e6162f7 (fix(docs,BOB-075): refresh docs/features/Status.md + docs/codegraph/Status.md staleness).
- **channel:** agent-code-reading — the subagent ran `codegraph init . --force` for real on this host (the doc's own documented Option-A mechanical-regen mechanism), observed the run walk **32,260 files / 514,456 nodes / 724,013 edges** (1.8 GB DB, still growing) before deliberately aborting it — a **~63× blowup** versus the 2026-06-06 baseline of 509 files / 8,906 nodes for the same repository shape — then root-caused it by direct inspection, not by any standing test or HelixQA run. Full evidence + before/after headers: docs/codegraph/Status.md Revision 2 (lines 87-124) and docs/qa/BOB-075/{before_state,after_state}.txt.
- **escape-audit:** root cause CONFIRMED (not guessed, §11.4.6) via `git check-ignore -v: nested frontend/.gitignore:10 (/node_modules)` and `extension/.gitignore:2 (node_modules/)` both correctly exclude their respective node_modules/ trees for git itself (`git ls-files frontend/node_modules extension/node_modules` → 0 rows both), yet `codegraph init` on CodeGraph 1.5.0 walked into both trees anyway (365 MB + 236 MB combined) — a CodeGraph 1.5.0 regression in honoring **nested** (non-root) .gitignore files, not honoring the same exclusion path git itself respects. frontend/ and extension/ were added to this project after the 2026-06-06 CodeGraph setup and had never been exercised against this exclusion path before — **genuinely uncovered:** no automated check exists anywhere in this repo that re-indexing honors nested .gitignore scope, and the tool's own documented “zero-config, exclusion driven by .gitignore” claim (the 2026-06-06 Status.md entry) was never re-verified after frontend// extension/ were added or after the tool moved from the documented 0.9.9 to the actually- installed 1.5.0.
- **new-check:** not yet authored. Followup filed: **BOB-104** — author a challenge (challenges/scripts/codegraph_gitignore_honor_challenge.sh or equivalent) with §11.4.115 RED_MODE polarity: RED_MODE=1 reproduces the blowup against the live nested-.gitignore tree, RED_MODE=0 asserts a resync stays within the documented baseline order of magnitude.

Corrective action for the underlying tool defect itself: file upstream at github.com/vasic-digital/codegraph (tracked as this ledger entry's new-check follow-through, same scope as BOB-104).

- **Honest boundary (§11.4.6):** the run was aborted (SIGKILL) per §12.6/§12.11 host-resource discipline before it could complete or corrupt state — no partial index was left claiming to be valid (codegraph status confirmed “index truncated” post-kill, and the truncated `.codegraph/codegraph.db*` is gitignored, never tracked). This project's CodeGraph index is therefore **NOT live** as of this entry; the last VALIDATED sync remains the 2026-06-06 baseline (7 PASS / 0 FAIL, pre-frontend/extension tree shape).

FORCED-LOGOUT-2026-08-18-2ND — user@1000.service SIGKILLed at 20:50:59, cascade kill of user session, operator perceived as forced logout

- **id:** BOB-116 (this-session incident; corrected 2026-08-18 from an informal “BOB-076” session-commit label that collided with a distinct, pre-existing, unrelated DB item — BOB-076 is Type=Task, the RD2-09 jackett fork bump, minted 2026-08-15 by commit 99a486e — per §11.4.54 no-ID-reuse; see the NOTE at the top of docs/incidents/2026-08-18-perceived-forced-logout-2nd.md and closure commit 2861920. followups tracked as Task #77 [wire challenge] / Task #78 [host_no_auto_poweroff_challenge perf regression] / Task #79 [SIGKILL-source attribution long-run]).
- **date:** 2026-08-18 — the 2nd forced-logout on this project (1st was 2026-07-07, produced §12.12 anchor); operator sent a CRITICAL mid-turn message reporting they had been “fully logged out from host milosvasic account” after returning from lid-closed state to find themselves at the GDM greeter.
- **channel:** operator-report — the incident was discovered by the operator observing the physical state (logged out at the GDM greeter after lid re-open at 20:51:06). No standing check monitored the resource-pressure signatures that preceded it (§12.12 EAGAIN cascade at 20:45:48, 15 GB pathological ugrep from Task #52 subagent, HTTP flood at 20:49:00, multi-fleet concurrent container pressure). systemd itself logged the user@1000.service: Main process exited, code=killed, status=9/KILL but no gate consulted that log to escalate before session termination. Full evidence: docs/qa/BOB-116/{journalctl_20-40_to_20-52.log, oomctl_snapshot.log, psi_readings.log, ps_LRSS_snapshot.log, challenge_pass.log, challenge_polarity_forced_fail.log, const033_challenge_pass.log, lid_and_session_events.log} (initially referenced as BOB-076 informal label, corrected

2026-08-18) and docs/incidents/2026-08-18-perceived-forced-logout-2nd.md.

- **escape-audit: root cause CONFIRMED as UNCONFIRMED** (§11.4.6 honest — the exact mechanism that delivered SIGKILL to user@1000.service is not attributable from the systemd journal; kernel OOM never fired, systemd-oomd never triggered, CONST-033 forbidden mechanisms never invoked, HandleLidSwitch was ignore). **Contributing factors CONFIRMED:** (i) §12.12 thread-exhaustion EAGAIN cascade at 20:45:48 — jackett SocketException (11) Resource temporarily unavailable to iptorrents+kinozal+rutracker simultaneously, exactly 5 minutes before the SIGKILL; (ii) 15 GB pathological ugrep from Task #52 subagent (ugrep -o with .\{0,120\} variable-length context + 3-way alternation against 14K-line CLAUDE.md) — reaped post-relogin, freed 16 GB immediately (PSI Avg10 1.77 → 0.08); (iii) multi-fleet concurrent pressure (boba stack + sibling “shlomi” claude session’s helix-* stack + lava-postgres + 6+ MCP servers + Yandex + JetBrains + ollama on one user.slice). NO automated check existed anywhere in this repo that (a) monitored user@1000.service health, (b) detected the leading §12.12 EAGAIN-cascade signature before the crisis window, (c) capped subagent grep -o variable-length-context patterns against multi-MB files. Also FOUND during triage: no_suspend_calls_challenge.sh was pre-existing FAIL from false-positive on scratchpad/ + .superpowers/sdd/ files that CARRIER the rule text (§11.4.201(1) false-positive-refusal class) — a separate coverage escape of its own, fixed same commit.
- **new-check:** challenges/scripts/resource_pressure_signature_challenge.sh (NEW, committed as 1f42357) — 5-signature proactive detector: SIG-1 process >5 GB RSS (forensic FACT: 15 GB ugrep), SIG-2 thread util >70% of ulimit -u (§12.12 crisis: 95%), SIG-3 EAGAIN cascade in container logs last 15min (this incident: 4 trackers @ 20:45), SIG-4 user.slice PSI full avg60 >50 (half the systemd-oomd 90% trigger), SIG-5 pathological-regex cmdline (ugrep -o + .\{N,M\} + \| alternation) — PASS on clean state, FAIL when thresholds crossed. Live evidence in docs/qa/BOB-116/challenge_{pass,polarity_forced_fail}.log (initially referenced as BOB-076 informal label, corrected 2026-08-18). Not yet wired into pre_build_verification.sh nor a systemd-user timer — **Task #77** owns that wiring (challenge exists as a runnable artifact, not a scheduled invariant, until #77 lands). Also **corrective action for CONST-033 challenge false-positive:** scripts/host-power-management/check-no-suspend-calls.sh extended

EXCLUDE_PATHS to skip scratchpad/ + .superpowers/sdd/ — challenge returns to PASS.

- **Correction (§11.4.209 independent review, .superpowers/sdd/task-review-457cca4-a7e55f9-report.md, IMPORTANT-1; remedied this-session, task #78):** the original “§11.4.115 polarity verified” claim above was a §11.4/§11.4.6 metric-layer bluff — the only forced-fail evidence at the time (challenge_polarity_forced_fail.log) ran with SIG1_MAX_PROC_RSS_GB=0, a THRESHOLD mutation that trips on every process, proving the comparison operator works but not that the detector catches the ACTUAL pathological state; SIG-2/SIG-3/SIG-4/SIG-5 had no polarity evidence at all. Fixed forward: five REAL per-signature RED fixtures now live under challenges/fixtures/resource_pressure/ — a genuine >5.5 GB page-resident process (SIG-1), a genuine >70%-of-(subshell-lowered)-ulimit thread-utilization ratio measured against the REAL live thread count (SIG-2), a real ephemeral podman container emitting a real 4-hit EAGAIN cascade read via the real podman logs path (SIG-3), a genuinely-high (avg60=65.00) PSI reading injected via a new PSI_FILE override point exercising the detector’s real parse+compare code — real memory-pressure induction was deliberately avoided on host-safety grounds, see that fixture’s header (SIG-4), and a live process whose real /proc/<pid>/cmdline genuinely matches the detector’s own regex, verified byte-for-byte against that regex extracted from the challenge script itself (SIG-5). bash challenges/scripts/verify_resource_pressure_polarity.sh runs all five against the UN-MUTATED challenge and its DEFAULT thresholds — real run: RED confirmed: 5 / 5 FAIL: 0 SKIP: 0. Evidence: docs/qa/task-78/{sig1..sig5}_real_fixture_output.txt + docs/qa/task-78/verify_resource_pressure_polarity_output.txt.
- **Honest boundary (§11.4.6):** the SIGKILL source is **not** claimed identified — the journal shows the receipt of SIGKILL but the sender is not attributed, filed as PENDING_FORENSICS (Task #79). Killing the pathological ugrep is a bandage that fixes ONE known contributor class, not proof it was THE cause. The new challenge captures signatures the incident had, not a proof the next incident will be prevented — that requires the pre-build integration + systemd-user timer of #77 firing, catching an SIG-* over threshold, and escalating before user@1000 dies.

DB-BLOB-COMMITTED-WITHOUT-DELTA-3520621 — docs/workable_items.db binary blob committed with only a prose claim, no differential evidence

- **id:** none pre-existing (a §11.4.209 code-review finding, not an operator/user-reported defect) — remedy tracked as this-session Task #79 (`.superpowers/sdd/task-79-report.md`; note the SAME numeral “#79” is ALSO used elsewhere in this ledger’s own FORCED-LOGOUT-2026-08-18-2ND entry above for a DIFFERENT task [“SIGKILL-source attribution long-run”] — an honest, unresolved task-numbering collision in this session’s own SDD dispatch records, surfaced here rather than silently reconciled per §11.4.6; out of scope for this entry’s remedy to fix).
- **date:** 2026-08-18 — found during the independent §11.4.209 review of commits `457cca4..a7e55f9` (`.superpowers/sdd/task-review-457cca4-a7e55f9-report.md`, finding IMPORTANT-2).
- **channel:** agent-code-reading — found by an independent reviewer comparing `git show --stat 3520621` (an 8 KiB `docs/workable_items.db` blob change) against the commit message’s own claim (“meta table content is unchanged”) and cross-referencing the SAME session’s Task #41 investigation, which had independently confirmed shared-checkout races on this exact file. No test, gate, or HelixQA run flagged this — the review was reading the commit itself.
- **escape-audit: genuinely uncovered** — no automated check anywhere in this repo, at the time commit 3520621 landed, ever opened `docs/workable_items.db` and diffed its logical content across a commit boundary. The ONLY existing SSoT-integrity check (`workable-items validate`) asserts internal consistency of the CURRENT state; it has no concept of “what changed since the parent commit” and therefore cannot catch an under-disclosed mutation riding inside an otherwise-legitimate binary-blob commit. `commit-push-all.sh`’s `--scope` safety check (task #66/BOB-068) also cannot see inside the blob — it only verifies which FILES are staged, not what changed WITHIN one of them.
 - **Backfill result (this entry’s own finding, task #79):** running the new `scripts/capture-workable-items-db-delta.sh 3520621` helper against the real repository produced `docs/qa/db-deltas/3520621866e071a6d10ec06e5b432188fdac7129.diff`, confirming:
 - (1) items table — exactly TWO rows changed: B0B-108 (Queued→Fixed (→ Fixed.md), the commit’s stated, intended mutation) and B0B-104 (Queued→In progress, body expanded with an upstream-issue-filing update) — **the BOB-104 change is real, legitimately audit-trailed (item_history row 89, By='AI',**

timestamped 19:16:36, one minute before BOB-108's own row 90 at 19:17:47), and consistently reflected in the committed docs/Issues.md at that same commit — but it is NOT mentioned anywhere in the 3520621 commit message, which describes only the BOB-108 mutation and the surfacing of brand-new items BOB-109..BOB-114. (2) item_history — exactly 2 new rows (89, 90), both correctly attributed. (3) doc_segments — exactly 2 rows changed, consistent with BOB-108's segment relocating from the Issues doc to the Fixed doc. (4) meta, logic_groups, test_diary, test_diary_summary, obsolete_details, operator_block_details, firebase_metadata — byte-for-byte unchanged, confirming the commit message's "meta table content is unchanged" claim was TRUE as far as it went. **Verdict: no data corruption, no silent data loss, every row-level change is legitimately audit-trailed — but the commit message's disclosed scope was narrower than its actual content** (an under-disclosed-but-benign mutation, exactly the class §11.4.226 requires runtime/artifact-class evidence to rule out rather than accept on a source-class prose claim alone).

- **new-check:** two-part remedy, both landed this session (task #79): (1) scripts/capture-workable-items-db-delta.sh (NEW) — produces a differential SQLite dump (per-table row counts + full meta dump + unified .dump diff) between any commit touching docs/workable_items.db and its parent, written to docs/qa/db-deltas/<full-sha>.diff; §11.4.115 RED→GREEN polarity confirmed via a real §1.1 paired mutation (breaking the sqlite3dump call in a scratch copy of the script produces exit 127 and **zero files** at the real output path — the atomic temp-file-then-mv publish pattern was added specifically so a genuine capture failure can never leave a partial, misleadingly-"clean" file behind; the unmodified script then reproduces the full evidence cleanly). (2) scripts/commit-push-all.sh stage 5.5 (NEW) — automatically invokes the helper immediately after ANY future commit that touches docs/workable_items.db lands, and lands the resulting delta as an immediate scoped follow-up commit (docs(qa,db-delta): capture differential dump for HEAD <sha>) travelling to every upstream in the same run — so this specific gap cannot recur silently going forward. See docs/scripts/capture-workable-items-db-delta.md for the full mechanism.
- **Honest boundary (§11.4.6):** this remedy makes future docs/workable_items.db-touching commits self-evidencing; it does NOT retroactively re-open or re-litigate 3520621 (no history rewrite, §11.4.113) and does NOT claim the BOB-104 under-disclosure was malicious or harmful — the backfilled evidence shows it was

benign and fully audit-trailed. It IS, per this ledger's own purpose, a real coverage escape: a commit message described a narrower scope than what it actually committed, and no automated check existed to catch that gap before an independent review found it by hand.

FORCED-LOGOUT-2026-08-18-3RD — user@1000.service SIGKILLED at 23:45:49 (3rd occurrence), plus the preventive monitor's own architectural gap

- **id:** BOB-120 (Type=Bug, Severity=Critical, Status=Queued — this document doc + evidence capture do not close it; closure requires the actual out-of-user-scope watchdog fix, tracked as a followup of task #77). Full writeup: docs/incidents/2026-08-18-3rd-forced-logout.md.
- **date:** 2026-08-18, 23:45:49 — the 3rd occurrence of this incident class on this host (1st: 2026-07-07, produced \$12.12; 2nd: BOB-116, same day 20:50:59; 3rd: this entry, ~2h55m after BOB-116, same session lineage).
- **channel:** agent-code-reading / session-continuation-dispatch — this incident was investigated proactively by an agent picking up a mid-turn task-continuation dispatch, not by the operator manually re-reporting a fresh logout observation this time. The real systemd journal, podman ps/podman logs, and the BOB-116 preventive timer's own fire history were read directly (docs/qa/BOB-120/{journalctl_23-42_to_23-46.log, timer_fire_history.log, qbittorrent_proxy_socketserver_traces.log}).
- **escape-audit:** the SIGKILL-source question remains the SAME open escape as BOB-116 (Task #79, PENDING_FORENSICS, no automated check exists anywhere in this repo that attributes a user@1000.service SIGKILL to its sender) — this incident adds a **second, distinct** coverage escape: the exact preventive check BOB-116 built in response to the 2nd occurrence (boba-resource-pressure-check.timer, task #77) is **architecturally incapable of catching a kill that lands inside its own hosting scope**, because the timer is a systemd --user unit whose process tree is a child of user@1000.service itself. When that service is SIGKILLED, the timer + its triggered probe die in the exact same cascade — the monitor cannot outlive the thing it monitors, and by construction can never observe or escalate on a kill that happens to land in the window between its own hourly fires (up to ~1h05m per cycle, base 1h + up to 5m jitter). Captured fire history shows the last completed run before this incident finished at 22:57:58 — 47m51s before the 23:45:49 kill — so the timer had not even reached its next

scheduled activation; this was not a missed fire, it was a structurally-unreachable one. This is a genuine, previously undocumented limitation of the BOB-116 remediation, not a bug in the remediation's own logic (the 5-signature detector itself is unchanged and still self-validated per §11.4.107(10) from BOB-116/task #78).

- **new-check: NOT YET LANDED** — this entry documents the finding + files BOB-120 as the tracked follow-up; the actual fix (an out-of-user-scope watchdog: a root-level systemd timer, or a --user unit escaped from the session-scope cgroup via delegation/isolation) is deliberately left as open, Queued work rather than rushed into this incident-response window. Closing BOB-120 on documentation alone, without the architectural fix landing, would itself be a §11.4.238 coverage-escape bluff — the exact failure mode this anchor exists to forbid.
- **Honest boundary (§11.4.6):** the originally-circulated hypothesis for this incident (“the timer’s next expected fire was 23:42:38 and it silently missed it”) does **not** survive contact with the real captured journalctl --user fire history (last completed run 22:57:58, next due no earlier than 23:52:58) and is explicitly corrected in the incident document rather than repeated as fact. The qbittorrent-proxy ConnectionResetError cascade (every ~30s through 23:38–23:45, last line 5s before the kill) and the boba-stack containers’ near-instant re-creation (CreatedAt=23:45:52, 3s after the kill) are reported as time-correlated observations, not established causes.

FORCED-LOGOUT-2026-08-19-5TH — 5th user@1000.service SIGKILL at 15:28:22, architectural install-gap: 4 authored preventive gates, 0 installed

- **id:** BOB-124 (Type=Bug, Severity=Critical, Status=In progress — filed this session, evidence at docs/qa/BOB-124/incident-5-forensics.log). Companion to BOB-116/BOB-120/BOB-123 (incidents #2/#3/#4); full mechanism/triage lives in the forced-logout-incidents project-memory playbook.
- **date:** 2026-08-19, 15:28:22 — the 5th forced-logout on this project (fresh host boot 15:07 → kill 15:28, exactly 21 minutes into a fresh session, first physical logout since incident #4’s ~00:37 occurrence).
- **channel:** operator-report at initial detection (operator physically observed the GDM greeter after lid re-open), corroborated by agent-code-reading during triage (journal grep confirmed the same PAM-session-close-synchronicity signature already documented across #2/#3/#4). No standing automated check monitored either

the SIGKILL delivery or the leading resource-pressure signatures during this session. Evidence: docs/qa/BOB-124/incident-5-forensics.log.

- **escape-audit: the coverage escape is now architectural, not per-check** — this is the 5th consecutive incident of the same class and the underlying block is the same across all five: the preventive gates authored in response to incidents #2/#3/#4 (BOB-116 5-signature detector, BOB-120 out-of-scope watchdog design, BOB-123 PAM-session-close monitor, sundry auditctl kernel rulesets) ALL require a `sudo/su -c` install step that has never actually been executed by the operator. The gate text ships; the mechanism does not run. Per `superpowers:systematic-debugging skill Phase 4.5` (≥ 5 attempts against the same block = architectural problem, not hypothesis failure) and per §11.4.250 (heuristic-tower signals a primitive defect), authoring a 6th detector would raise the tower height without raising the ceiling — the coverage escape is the install-gap, not any missing signature. The `boba-resource-pressure-check.timer` from BOB-116 DID install (it is a `--user` unit, no `sudo` needed) but is architecturally blind to a kill of its own hosting scope (BOB-120 gap), so its installed but blind status does not close this escape either: `existed-but-uninstalled` (a distinct §11.4.201(6) FALSE-NULL sub-class from `existed-but-missed` — the check exists in text, but no live instance runs to observe the defect at all).
- **new-check: NOT AUTHORIZING A NEW CHECK.** Followup filed as BOB-124 itself: install one of the already-authored preventive paths (Path 1 kernel auditctl rules OR Path 2 out-of-scope systemd watchdog OR Path 4 dedicated boba-watchdog UID) via an operator `sudo` session, and verify it is live via `auditctl -l non-empty` or `systemctl list-units --system 'boba-watch*' non-empty` before this ledger entry may be closed. Closing this entry on a 5th authored-but- uninstalled detector would itself be a §11.4.238 coverage-escape bluff (the exact failure mode this anchor exists to forbid) AND a §11.4.250 heuristic-tower violation.
- **Honest boundary (§11.4.6):** the SIGKILL initiator remains UNCONFIRMED across all 5 incidents — the PAM `session_close` correlation is the mechanism, not the root initiator, and `Linger=yes` should have prevented the kill by design (whether a systemd bug, an unattributed `loginctl terminate-user` call, or an SDD/container-fleet component invoking a stop path at a different identifier we have not yet grepped is still open). This entry does NOT claim a root cause; it claims the coverage escape is the install-gap on the ALREADY-KNOWN preventive paths, which is itself proven by 5 consecutive occurrences without a single mechanical detection.

COMPUTE-BADGES-CARRIER-MATCH — `compute-badges.sh` destroyed a README prose line (substring match, not structural) and accumulated a blank line in `docs/TESTING.md` on every run

- **id:** COMPUTE-BADGES-CARRIER-MATCH (no BOB-NNN minted — fixed in the same session it was found; slug per the §11.4.238 schema’s “else a short slug”).
- **date:** 2026-08-20
- **channel:** agent-code-reading — found while reviewing the git diff produced by running `scripts/compute-badges.sh` as the prescribed remediation for a CM-BADGE-FRESHNESS-CHECK WARN. The corruption was visible only because the diff was read line-by-line before committing.
- **summary:** Two distinct defects in `scripts/compute-badges.sh`. (1) CORRUPTION: the README rewrite used bare awk patterns `/alt="tests"/` and `/alt="vitest"/`, which fire on ANY line containing those literals — including the prose paragraph at `README.md:56` that *documents this very filter*. The script replaced that prose line with an `<img>` badge tag, destroying documentation content. (2) NON-IDEMPOTENCE: the `docs/TESTING.md` section regenerator stripped the old `## Test counts` section but NOT the blank line(s) preceding it, then unconditionally re-emitted a leading blank line — accumulating one blank line per run without bound. The live file had reached 7 accumulated blank lines, i.e. the defect had been running silently for ~7 regenerations.
- **escape-audit:** CM-BADGE-FRESHNESS-CHECK (invariant 26 in `scripts/pre_build_verification.sh`) was the only standing check over this script, and it verifies **only that the badge COUNTS match live counts** — it never asserted that the rewrite left non-badge content intact, so a rewrite that corrupts prose while producing correct badge lines passes it. Worse, the checker is *structurally incapable* of seeing this class: `compute-badges.sh --check tests grep -qF "${PY_LINE}"`, and the corruption REPLACES the prose line with exactly `${PY_LINE}` — so on a corrupted README the checker reports “in sync” (a §11.4.201(6) FALSE-NULL in the checker itself). No check anywhere asserted idempotence, which is why defect (2) accumulated 7 times undetected. This is the §11.4.201(7)(a) carrier-match footgun (match STRUCTURE, not substring) reproducing inside our own tooling — the same class already anchored twice (§11.4.196(D) process-carrier, §12.12 self-match).
- **new-check:** `tests/unit/test_compute_badges_carrier_match.sh` (6 assertions), which drives the REAL script through its `REAL --readme/--testing-md` invocation path (§11.4.201(11)) against a

fixture containing both a genuine badge block and a carrier prose line. §11.4.115 RED-capture evidence, observed before the fix: FAIL: carrier prose line was OVERWRITTEN – substring match instead of structural match with the fixture showing the prose line replaced by `<img alt="tests" ... python%20tests-5358%20collected-blue">`, and FAIL: TESTING.md: blank lines ACCUMULATE across runs (2 -> 3) – not idempotent. GREEN after the fix: RESULT: 6 passed, 0 failed, with the real README.md verified byte-identical (prose survived) and the real docs/TESTING.md self-healing 7 → 1 blank lines. The test also carries a false-negative guard (assertion 2 fails if a “fix” simply disables the rewrite, §11.4.201(1)).

GITIGNORE-SHADOWS-TRACKED-FILES — 58 TRACKED files were rejected by `git add`, silently breaking the §11.4.234 commit mechanism

- **id:** GITIGNORE-SHADOWS-TRACKED-FILES (no BOB-NNN minted — fixed in the same session).
- **date:** 2026-08-20
- **channel:** agent-code-reading — surfaced when `scripts/commit-push-all.sh` aborted at stage 5 with The following paths are ignored by one of your `.gitignore` files: `.specify/memory` while committing the tracked project constitution. A repo-wide sweep then generalised the instance.
- **summary:** When a `.gitignore` rule matches a path that is ALSO tracked, `git add <path>` exits NON-ZERO. Under `set -euo pipefail` that kills the mandated §11.4.234 commit entrypoint mid-run — the file is committed in the repo and visibly modified in `git status`, yet impossible to commit through the only sanctioned path. A sweep found **58** such tracked files across **five** rules: `.claude/` (the §11.4.109 PreToolUse hook config), `.opencode/` (15 Spec Kit agent commands), `.specify/*` (32 governance/template files), `config/merge-service/` (3 production config files), `.playwright-mcp/` (8 generated snapshots). Notably `.gitignore:206` already carried `!.claude/settings.json` — an INERT negation, because `git` cannot re-include a file whose parent directory is excluded (`dir/` must be `dir/*` first). The author’s intent was correct and had been silently defeated for an unknown period.
- **escape-audit:** NO check existed over the tracked-vs-ignored intersection at all — not in `scripts/pre_build_verification.sh`, not in `scripts/commit-push-all.sh` (whose stage-2 cheap validation checks `.env` leakage but never `git add`-ability), and not in any test. The condition is invisible in normal operation because it only

manifests when a shadowed file is *modified*; all 58 sat dormant. The §11.4.234(D) always-unblocked invariant had no mechanical enforcement, and §11.4.30 (“tracked authoritative content is never gitignored”) existed only as prose — the §11.4.227 prose-does-not-bind/seams-do failure verbatim.

- **new-check:** tests/unit/test_tracked_files_are_addable.sh.
Enumerates rule-matched tracked paths in one `git check-ignore --no-index` pass, then verifies each through the REAL `git add --dry-run` invocation path (§11.4.201(11)), and carries a **control needle** asserting the detector can actually see (§11.4.201(7)(b) — a blind instrument and a clean repo both return a quiet zero).
§11.4.115 RED-capture evidence, observed before the fix: FAIL: 50 TRACKED file(s) are rejected by 'git add' — the §11.4.234 commit mechanism is broken for them, each named with its responsible rule, alongside PASS: control needle: check-ignore instrument is seeing. GREEN after the .gitignore negations: RESULT: 2 passed, 0 failed. The 8 .playwright-mcp/ entries are held in a documented exclusion fence (§11.4.224(E) style) rather than silently un-ignored: they are tracked GENERATED artifacts whose correct remedy is to stop tracking them (§11.4.30), a removal that is an operator decision under §11.4.122 requiring the §11.4.124 git-history investigation first — recorded as TODO(PLAYWRIGHT-MCP-ARTIFACTS) in the test.
- **Honest boundary (§11.4.6):** the fix un-ignored the tracked trees and the guard prevents recurrence repo-wide, but it does NOT retroactively prove the 58 shadowed files were otherwise correct — only that they are now committable. Un-ignoring also revealed the vendored .specify/extensions/superspec/ checkout (own .git gitdir pointer, ~14 MB assets, its own .github/workflows/ci.yml, duplicating the root superspec submodule); it was deliberately RE-ignored rather than committed, since committing it would mint a phantom gitlink absent from .gitmodules.

NON-HERMETIC-TESTS-MUTATE-REAL-TREE — a dead session-scratchpad path made one test a FALSE PASS, and its plain-cp restore bumped mtimes that failed two OTHER suites

- **id:** NON-HERMETIC-TESTS-MUTATE-REAL-TREE (no B0B-NNN minted — fixed in the same session).
- **date:** 2026-08-20

- **channel:** agent-code-reading — surfaced by the brand-new CM-BASH-UNIT-TESTS-EXECUTED invariant (itself added the same day), then root-caused with superpowers:systematic-debugging.
- **summary:** Three “independent” failing suites had ONE root cause. tests/unit/test_pre_build_workable_items_diff_check.sh (a) wrote and sourced its invariant-extraction helper at a HARDCODED agent-session scratchpad path under a DEAD session id (.../aa7d8260-.../scratchpad/inv17_extract.sh). Once that session ended both the redirect and the source failed, so the extractor emitted NOTHING — which made its assertion 2 a genuine FAIL and, far worse, made assertion 1 a **FALSE PASS**: empty output matched no “divergence” pattern, so a blind instrument reported “the current tree is in sync”. (b) It backed up and restored the REAL docs/Issues.md with a plain cp, which returns byte-identical CONTENT but a NEW mtime. CM-MARKDOWN-EXPORT-SYNC compares mtimes, so the restore manufactured “docs/Issues.html stale” and failed tests/test_constitution_inheritance.sh, which asserts the whole gate returns 0. The same plain-cp pattern existed in that suite too, for CLAUDE.md and constitution/Constitution.md. Measured cascade: docs/Issues.md 1787213360 → 1787213787 with ZERO content change. The third “failure”, test_export_sync_gate.sh, was never broken at all — it is state-dependent and had been observed while the tree genuinely had stale exports.
- **escape-audit:** The entire tests/unit/*.sh bash suite was executed by NOTHING — ci.sh runs pytest tests/unit/ which collects only test_*.py, run-all-tests.sh only bash -n syntax-checks, and pre_build_verification.sh merely MENTIONED some suites in comments. So no seam ever ran them, and §11.4.226’s “registration is not coverage” held exactly: these defects sat dormant for an unknown period. Nothing anywhere asserted that a test leaves the real tree unmodified (§11.4.84 quiescence had no mechanical enforcement), and nothing asserted that an extraction helper actually produced output before its silence was read as a clean result (§11.4.201(6) FALSE-NULL, §11.4.201(7)(b) missing control needle).
- **new-check:** THREE seams, each mutation-proven. (1) Invariant [30/30] CM-BASH-UNIT-TESTS-EXECUTED in scripts/pre_build_verification.sh now RUNS the suite (7 green, 0 quarantined). (2) That invariant gained a **NO-TRACE assertion**: it snapshots the mtimes of docs/Issues.md, CLAUDE.md and constitution/Constitution.md before the suite and FAILs if any moved. §1.1 paired mutation, run live: reverting one restore to a plain cp produced FAIL: a bash test MUTATED the real tree (mtime moved): docs/Issues.md (1787213787 -> 1787215890), gate

exit=1; restoring cp -p returned no-trace verified, gate exit=0, 32 passed / 0 failed. (3) A **control needle** in the diff-check suite now requires the extracted invariant to emit one of its own
PASS_MARKER:/FAIL_MARKER: lines before “no divergence” may be read as good news — the exact guard that would have caught the false pass. Post-fix: all three suites green (diff_check 2/0, export_sync_gate 3/0, constitution_inheritance 21/0), and constitution_inheritance runtime fell from a >840s timeout to 143s once its three nested gate invocations were tagged
BOBA_PREBUILD_NESTED=1.

- **Honest boundary (§11.4.6):** the no-trace assertion covers the THREE files these suites are known to mutate, not every tracked file — a suite that mutates something else would still slip through. Widening it to the whole tracked corpus is cheap but was not measured here, so it is stated as a gap rather than claimed. The deeper design issue also remains open: §11.4.86 mandates content-hash change detection and CM-MARKDOWN-EXPORT-SYNC still uses mtime, which is why a content-neutral rewrite can fake staleness at all. TODO(EXPORT-SYNC-CONTENT-HASH).

EXPORT-SYNC-MTIME-NOT-REPRODUCIBLE-ON-FRESH-CLONE — invariant 16 refuses a provably clean tree after git clone

- **id:** EXPORT-SYNC-MTIME-NOT-REPRODUCIBLE-ON-FRESH-CLONE (no BOB-NNN minted; NOT fixed — see honest boundary).
- **date:** 2026-08-20
- **channel:** agent-code-reading — found by a subagent investigating whether CM-MARKDOWN-EXPORT-SYNC could migrate from mtime to §11.4.86 content-hash detection.
- **summary:** Invariant 16 decides staleness with `[[ "${sib}" -ot "${md}" ]]` — a NANOSECOND-precision mtime comparison — over a 143-file corpus. **Git does not preserve mtimes**, and on checkout .html sorts before .md, so the write order races at millisecond resolution. Measured on two fresh git clones of this repo at the SAME commit, zero content drift: clone A reported **15** in-scope exports STALE, clone B reported **19**, with deltas of 1.000–2.000 ms (e.g. docs/features/Status.html 2.000 ms older than its .md). The gate passes on this working copy only because its exports happen to have been written after their sources. So invariant 16 is non-reproducible across checkouts (§11.4.50) and would REFUSE a provably clean tree on any fresh clone or CI checkout — the §11.4.201(1) false-positive FAIL-bluff class.

- **escape-audit:** Nothing ever ran invariant 16 against a FRESH CLONE. Every execution has been on a long-lived working copy where export mtime happen to trail their sources, so the failure mode is invisible by construction here. §11.4.86 explicitly mandates content-hash detection (“NOT mtime”) and this invariant has used mtime since it was written; the mandate existed as prose and no seam enforced it (§11.4.227 prose-does-not-bind). The content-hash oracle that WOULD be immune already exists and already runs — invariant 24 CM-DOCS-CHAIN-ENGINE-VERIFY — but covers only **3 of 143** authored .md sources (2%), and nothing measured that coverage gap.
- **new-check:** NONE YET — deliberately. The subagent probed the docs_chain engine in an isolated scratch root and confirmed it is immune to exactly this cascade (mtime-only bump → in-sync, exit=0; real content change → STALE, exit=1; missing state → fail-closed), but a migration needs ~140 new context nodes AND a regenerator wired into the export pipeline. Without the regenerator every legitimate doc edit would leave the baseline stale and the gate permanently red — the half-migrated state that is WORSE than today’s. Recommended order, to be tracked:
 1. extend .docs_chain/contexts/ to cover invariant 16’s corpus; (2) make the baseline reproducible (tracked hash manifest regenerated+staged in the same commit per §11.4.12/§11.4.106, or a documented fresh-clone sync); (3) ONLY THEN demote invariant 16 to existence-only and let invariant 24 own staleness. Doing (3) first would remove the only staleness signal covering 140 docs. TODO(EXPORT-SYNC-CONTENT-HASH).
- **Honest boundary (§11.4.6):** this entry records a defect that is **NOT fixed**. Invariant 16 is untouched. Recording it here with no new-check is the honest state — claiming a closure would be the §11.4.226 evidence-class bluff. The fresh-clone measurement is real and repeatable; the migration is not attempted because it could not be proven safe within this change’s file scope.

BOB-CONTAINER-WRITES-NOT-OPERATOR-OWNED — every file the system produced was owned by an identity the operator did not have

- **id:** feature 002-user-owned-downloads (spec + tasks under specs/002-user-owned-downloads/); no single BOB-NNN mints the defect itself.
- **date:** 2026-08-21
- **channel:** operator-report — the operator hit it while using the product: downloaded content, and the project’s own config/ tree,

could not be moved, edited, or deleted by the account that started the system. Nothing in the automated regime raised it, before or after.

- **summary:** Every file the containerised services wrote landed at host uid 100999 — a rootless-container sub-uid with **no host account**. Measured on 2026-08-21 before the fix: the download root rendered as UNKNOWN:UNKNOWN and held content at 100999; config/ held **51** items at 100999; and config/boba.db (mode 600, owner unresolvable) could not be read by the operator at all — which silently made the backup procedure that docs/BOBA_DATABASE.md § 3 *mandates* impossible to perform. The document prescribed no workaround because it never knew it was blocked.
- **escape-audit: No automated check ever asserted the OWNERSHIP of a file the system produced.** This is a scope gap with a precise shape, not a missing idea. The suites that covered these paths asserted *completion* and *response status*: a download finished, an endpoint returned success, a file appeared where it was expected. Every one of those properties is **true of a file owned by 100999** — they stayed green for the entire life of the defect, and would have stayed green indefinitely, because none of them ever read a produced file's st_uid back from the host and compared it to anything. The defect was therefore invisible **by construction** to the whole standing regime, not merely missed by it. Two compounding factors, both §11.4.201-class: (a) ownership of a location **cannot be inferred from the location itself** — the download root was owned by uid 1000 and still received 100999-owned files, so any check reading the directory's own owner, the configured PUID, or “no error occurred” would have passed while broken (a proxy standing in for the condition); and (b) a **host-side** write by the operator produces an operator-owned file even in the directory whose *container* writes land at 100999 — measured during Phase 1 of the fix, where the shared host probe returned ok for exactly that directory. So even the obvious first version of the check would have been a false pass. The check that would have caught it is precisely the one that now exists: write a real file **through the real application path**, read the owner back **from the host**, and compare it to the operator's uid.
- **new-check:** four artifacts, all landed on branch 002-user-owned-downloads:
 1. tests/ownership/test_container_writes_owned_files.py — the §11.4.115 **RED**. A throwaway container writes a file and a directory tree into the location as the service's declared identity; the test reads the owner back from the host. Observed **failing at uid 100999** before the fix (the test's own docstring records RED

(pre-fix): owner uid is 100999 – an identity with no host account); passes now. Covers FR-002 at every level of a created tree, not only the leaf file.

2. `tests/unit/test_ownership_precondition.sh` — contract suite for `scripts/ownership_precondition.sh`, the fail-closed startup check that refuses to start when a declared location cannot produce operator-owned files. Includes the §11.4.201(1) both-directions set the contract mandates: golden-bad (wrongly-owned location → exit 1), golden-good (→ exit 0), and a negative control (an optional: true location that is simply absent → exit 0, **not** a refusal).
 3. `tests/unit/test_ownership_repair.sh` — contract suite for `scripts/ownership_repair.sh`, including the interrupt case (kill mid-run → marker absent → next run resumes) and the preserve-mode case (`config/boba.db` keeps its exact bits after repair).
 4. **CM-OWNERSHIP-INVARIANTS** — the standing gate, wired as **invariant 33** of `scripts/pre_build_verification.sh`, delegating to `scripts/pre_build/check_cm_ownership_invariants.sh` with its paired §1.1 mutation `check_cm_ownership_invariants_mutation_test.sh` (revert one service's ownership route → the gate must FAIL). This is what stops the route from being silently reverted later — per §11.4.226, a source-only fix with no standing guard is exactly the closure class that reopens the next time someone edits `docker-compose.yml`.
- **Honest boundary (§11.4.6):** the defect is fixed and the checks above are in place, but the escape itself is not undone — this is recorded as a **coverage escape**, not as a clean closure, because a human using the product was the discovery layer and §11.4.238 makes that a failure of the regime regardless of how well the defect was then fixed. Two residuals are stated rather than glossed: (a) the `start.sh` wiring (`run_ownership_gate()`, refusing to start on precondition exit 1 **or** 2 and on a failed repair) is present in the working tree as of 2026-08-21T15:10Z but was still **uncommitted** when this entry was written — so the gate is not yet proven by a run of the committed tree, and task **T031** remains the tracking item until it is; (b) the container-write probe inside `scripts/ownership_precondition.sh` needs a container runtime and a locally-present service image, and honestly SKIPS with a named reason where it has neither (§11.4.3) — on such a host the standing check verifies the declared ownership **route** (configuration) rather than observed **behaviour**, and says so in its own output rather than reporting the fallback as the real condition.

BOB-169 — three producers emitted charset-less HTML, weasyprint baked mojibake into the PDF text layer, and every export gate stayed green

- **id:** BOB-169 (status Queued in docs/workable_items.db as of this audit — the fix is in the working tree, the item is NOT closed)
- **date:** 2026-08-21 (census + root cause; docs/qa/BOB-169/root_cause_proven_20260821.md). Audit written 2026-08-25.
- **channel:** agent-code-reading — an agent opened an exported PDF and read mojibake in its text layer, then traced the generator in source. Invariant 50's own header records the same attribution: "The defect was found by an agent reading a PDF, not by the regime."
- **summary:** scripts/generate_markdown_exports.sh ran pandoc -f markdown -t html5 with no --standalone, so pandoc emitted a BODY FRAGMENT — no <!DOCTYPE>, no <head>, no <meta charset>. weasyprint then rendered the PDF **from that fragment**, fell back to a non-UTF-8 codec, and baked the mojibake into the PDF's **text layer** — corruption in the bytes, not a display artifact. Two further producers shared the defect (scripts/regenerate-continuation-exports.sh; the docs_chain engine's derived.go:139 already passed --standalone), and the third was found only because a corpus-wide scan was finally run. docs/CONTINUATION.pdf — the §12.10 session-resumption document a fresh session is pointed at — regenerated corrupt on every run. **Re-measured for this audit against HEAD's committed blob (git show HEAD:docs/CONTINUATION.pdf): 334 corrupted lines; the working-tree copy: 0.** The earlier evidence pack recorded 336; the two numbers come from different mojibake-signature definitions and are left un-harmonised rather than silently reconciled (§11.4.6). Corpus-wide, a live run of the gate on 2026-08-25 reports **301 of 334** generated exports carrying no charset declaration.
- **escape-audit:** The export regime asserted **PRESENCE and MTIME, never VALIDITY** — and this is readable as two predicates, not as a judgement. scripts/pre_build_verification.sh invariant 16 (CM-MARKDOWN-EXPORT-SYNC, §11.4.65) evaluates exactly:

```
if [[ ! -f "${sib}" ]]; then                                # (1) does
    the sibling EXIST?
elif export_is_stale "${md}" "${sib}" ...; then           # (2) is the
    SOURCE newer than it?
```

Neither predicate opens the file. A charset-less body fragment and a well-formed standalone document are indistinguishable to both: both exist, both are fresh. That is why 301 violations could accumulate under a gate whose name says “export sync” — it synchronises *existence and ordering*, and was never a content oracle. Verified by reading the produced bytes rather than trusting the count: the charset-less files begin `<h1 id="agents.md">` (AGENTS.html), `<h1 id="changelog">` (CHANGELOG.html), `<h1 id="claude.md">` (CLAUDE.html); the compliant ones begin `<!DOCTYPE html> ... <meta charset="utf-8" />`.

A content-aware oracle **did** exist and **did** run — invariant 24 CM-DOCS-CHAIN-ENGINE-VERIFY, which hashes content rather than comparing mtimes. It was simply almost empty: measured for this audit, `.docs_chain/contexts/` holds **2 contexts covering 4 authored .md sources** (`docs/codegraph/Status.md`, `docs/codegraph/Status_Summary.md`, `docs/features/Status.md`, `docs/features/Status_Summary.md`; an earlier ledger entry measured 3 — the corpus grew). Coverage and correctness coincide **exactly**: all four of those .html files carry `<meta charset="utf-8" />`, and the one producer that always passed `--standalone` (`derived.go:139`) is precisely the producer those four run through. Everything outside that 4-document island was governed only by existence-and-mtime.

Two compounding mechanisms explain the age and the silence, both measured rather than inferred.

1. **The generator is mtime-gated**: it rewrites an export only when the source is newer, so a fragment minted once persists indefinitely — the corpus accumulated its violations one document at a time, and no point exists at which anyone could say the corpus was done. (b) **The fallback path was the correct one**: the else branch (python-markdown, used only when pandoc is ABSENT) writes `<meta charset="utf-8">`. So a host *without* pandoc produced correct exports and a properly-provisioned host produced broken ones — the defect is invisible on exactly the machines least likely to be treated as degraded.

Recorded because it is the same shape one level up: **this ledger's own .html is one of the 301 charset-less fragments, and docs/QA_DISCOVERY_LEDGER.pdf carries 494 mojobake lines** measured during this audit. The document that tracks coverage escapes is itself downstream of the escape it records.

- **new-check: PRESENT AND EXECUTED, BUT NOT YET COMMITTED** — read the residual before treating this as closed. Four artifacts, all live in the working tree, all run on 2026-08-25 for this audit with the output pasted below:

1. `scripts/pre_build/check_cm_export_charset_valid.sh`, wired as **invariant 50 (CM-EXPORT-CHARSET-VALID)**. It matches a `<meta ... charset ...> ELEMENT`, never the bare substring — the naive `grep -qi charset` PASSED against the broken generator by matching a heading slug (§11.4.201(7)(a)). It carries its own control needle: `TOTAL == 0` or `COMPLIANT == 0` is a REFUSAL, so a blind enumeration can never report a clean zero. Brownfield adoption is a §11.4.135 monotone-decrease ratchet pinned at 301, so it fires on regression only. Live run:

```
generated exports scanned ..... 334
declaring a charset ..... 33
MISSING a charset ..... 301    (ratchet baseline
301)
PASS: CM-EXPORT-CHARSET-VALID – 301 charset-less exports,
at baseline (no regression)
```

2. `tests/pre_build/test_cm_export_charset_valid.sh` — the §1.1 paired mutations. Live run: `RESULT: 6 passed, 0 failed`, including both BLIND corpora (a naive gate reports zero and passes) and the §11.4.201(1) negative control (at baseline → PASS, no false refusal).
3. `tests/unit/test_export_pdf_charset_integrity.sh` — drives a COPY of the real generator against a temp tree and asserts the PDF **text layer** preserves the source’s non-ASCII. Live run: `RESULT: 2 passed, 0 failed` — “PDF text layer preserves the source’s non-ASCII (needle: 1 line(s) intact, 0 mojibake)”.
4. The three producers are now in lockstep on `--standalone`, and PDF staleness is keyed on the HTML (`|| "$html" -nt "$pdf"`) so a repaired HTML actually re-renders its PDF instead of leaving the old mojibake baked in.

- **Honest boundary (§11.4.6) — three residuals, none of them cosmetic:**

1. **The gate is not in the repository.** Measured with a needled instrument (the same `grep` finds `CM-OWNERSHIP-INVARIANTS 5×` at HEAD, so it can see): `git show HEAD:scripts/pre_build_verification.sh` contains **zero** occurrences of `CM-EXPORT-CHARSET-VALID` and its last invariant marker reads

[49/49]; the working tree reads [50/50]. scripts/pre_build/check_cm_export_charset_valid.sh, tests/pre_build/test_cm_export_charset_valid.sh and tests/unit/test_export_pdf_charset_integrity.sh are all **UNTRACKED**. A fresh clone at dd9fcb5 therefore has 49 invariants and no charset gate at all. Per §11.4.215 an untracked artifact does not bind, and per §11.4.226 a check that exists only in one working copy is the install-gap class this ledger already recorded once (FORCED-LOGOUT-2026-08-19-5TH: “4 authored preventive gates, 0 installed”). **Committing these four artifacts is the outstanding action; this audit does not perform it** (the tree is not quiescent and three sibling agents hold the surrounding files).

2. **Acceptance (c) — the bulk regeneration of the 301 charset-less exports — has NOT been run.** The mechanism exists and is correctly sequenced (HTML before PDF, or weasyprint rebakes the same mojibake), but the corpus is still 301 violations deep and heals only as documents are touched.
3. **BOB-182 is open:** the §11.4.224(E) ratchet-adoption decision is owed to the operator, and invariant 50 is already enforcing. Filing the item satisfies the tracking obligation, not the decision obligation.

BOB-168 — four destructive commands consuming a possibly-empty root, under a suite that passed 10/10 either way

- **id:** BOB-168 (status Queued in docs/workable_items.db)
- **date:** 2026-08-23 (docs/qa/BOB-168/round3_guard_residue_fix.md)
- **channel:** agent-code-reading — surfaced by the independent §11.4.209 code-review agent in round 2 (NO-GO), reading the test source; no test run, and no gate, reported it.
- **summary:** Four pre-run mutation lines in tests/unit/test_run_all_challenges_missing_entry.sh consumed a possibly-empty \$R **before** run_runner’s own guard could refuse — chmod, printf > (twice) and rm -f. On a failed fixture they operate on /submodules/...-rooted paths: the canonical §11.4.252 shape, an unguarded expansion inside a destructive command. Measured pre-guard residue, verbatim: chmod: cannot access '/submodules/challenges/...' plus two No such file or directory writes, and — proven later at the syscall layer with strace -e trace=unlink,unlinkat — one unlinkat("/submodules/...") = -1 ENOENT from the rm -f, the one line that leaves no shell-level trace at all. Post-guard: 3 → 0 write attempts, 1 → 0 unlinkat, with 16 unlink-

class syscalls captured in the same trace proving the instrument was watching (a *seen* zero, not a blind one).

- **escape-audit:** Four independent reasons the standing regime could not have raised this, each read from source or measured, none of them “we didn’t think of it”:

1. **Scope.** The §11.4.252 gate is wired as invariant 39 CM-DANGEROUS-COMBINATION-FAIL-CLOSED, and its inclusion list is `DANGER_ROOTS=(download-proxy/src plugins scripts qBitTorrent-go frontend/src)`. **tests/ is not in it.** In production the gate never looks at the directory where this defect lived.
2. **Pattern set.** Even pointed at `tests/unit` by hand for this audit it cannot see this shape. The gate implements exactly two detectors — an empty/comment-only `catch/except` body, and a credential defaulting to a literal. It has **no rule for a destructive command consuming a possibly-empty expansion**. Run against `tests/unit` it reports **17 hits, all .py, all of the except: pass shape, and zero naming test_run_all_challenges_missing_entry.sh** — a seen zero, needled by 50 finding lines from the same gate against `plugins/`.
3. **Severity.** Invariant 39 is **ADVISORY**: it emits `WARN`, never `fail`. Even a hit would not have blocked anything.
4. **The covering suite is blind by construction — this is the sharpest of the four.** The file `IS` executed by a standing gate (invariant 30 CM-BASH-UNIT-TESTS-EXECUTED runs `tests/unit/*.sh`), so this is not a registration gap. But every assertion in that suite is about the runner’s exit code and `MISSING` count — verbatim: “all entries absent -> expected exit 2”, “one entry not executable -> expected exit 2”, “a real challenge failure -> expected exit 1”. **Nothing asserts that no write escaped the fixture root.** The only occurrence of the word “residue” in the file is a comment on line 95 — a carrier, not an assertion (§11.4.201(7)(a)); verified by reading the line, not by trusting the count of 1. The consequence is recorded in the item’s own evidence: the suite scored “**GREEN — 10/10, unchanged**” both with the residue and without it. A guard never observed to `FAIL` on the genuinely-broken artifact is unvalidated instrumentation (§11.4.115(F)), and here the standing gate ran it, went green, and carried no information about the defect in either direction. Re-run for this audit: `RESULT: 10 passed, 0 failed, with / submodules confirmed absent`.

- **new-check: STILL OWED — no new automated check exists for this class.** What landed is a *source-level* fix, not a check: one shared predicate `usable_root() { [[ -n "${1:-}" ]&& -f "${1}/scripts/run_all_challenges.sh" ]};`, consumed by `run_runner` and by all

four mutation lines (now at 157 / 175 / 190 / 191). That is the right rung — §11.4.241 prefers an API-shape guard over four copies of an inline lint-level test, and §11.4.251 names the second, weaker inline notion of “valid root” as the mechanism that produced the residue — and a fifth mutation line added tomorrow inherits it. But **nothing would refuse a fifth line that does not**. The concrete owed work, in the order it should be done:

1. extend the §11.4.252 gate’s pattern set with the unguarded-expansion-in-a-destructive-command shape (`rm/chmod/mv/>` with a `${var}`-rooted path and no preceding non-empty test), shipping golden-TRUE and golden-FALSE-with-carrier fixtures per §11.4.201; (2) add tests to `DANGER_ROOTS`; (3) promote that specific shape from ADVISORY to blocking, leaving the untriaged except: pass backlog advisory so the promotion is not a §11.4.201(1) false-positive refusal. This audit **does not** make those edits — `scripts/pre_build_verification.sh` is held by a live sibling agent — so this is written as a finding, per the standing instruction.
- **Honest boundary (§11.4.6):** the pre-guard/post-guard comparison quoted above is the item’s own captured evidence, re-read for this audit but not re-executed by it (re-running it requires reverting the guard in a tracked file, which would leave residue). What this audit executed directly is the current suite (10/10), the §11.4.252 gate against `tests/unit` (17 hits, none this file) and against `plugins/` (50 hits, the needle).

BOB-164a — 21 WCAG AA contrast violations on the live dashboard, found by the axe suite on its first run

- **id:** BOB-164 (status Queued in `docs/workable_items.db`) — recorded as **164a**, the originally-filed defect, to keep it distinct from the round-1 regression at 164b below, which has a different discovery channel and a different audit.
- **date:** 2026-08-21T19:56:53Z (filing timestamp recorded in the item’s own Reported-Via field)
- **channel:** `automated-helixqa` — **the first entry in this ledger that is not out-of-band**. The item’s own filing records it verbatim: “**Reported-By:** BOB-110 UX-class coverage, discovered by the new axe-core suite on its first live run”, and “it was found by the automated regime rather than by a human squinting at the page — which is exactly the 11.4.238 posture the project is aiming for.” The failing test was deliberately **left failing** rather than silenced or `xfail`-ed, so a red UX suite reads as this item.

- **summary:** axe-core 4.13.0 against the shipped bundle in a real headless Chromium reported **21 colour-contrast violation nodes** on darcula/dark (plus 1 on the light theme, which the filing had not scanned). Root causes: `--color-accent` used as *text* (5 nodes, 1.24:1 declared), `--color-text-secondary` (11 nodes), `--color-text-primary` on `--color-bg-tertiary` (3 nodes), and two `opacity: 0.65` multipliers dragging compliant tokens below the floor. No failing node used inline hex — every one resolved through a `var(--color-*)` token, so the fix belonged at the token layer: `accent/danger/warning` were **split into disjoint text roles** (§11.4.217) rather than repainting a brand-locked colour. The all-palette scan — the first ever run — exposed a second, pre-existing class: `color: #fff` hardcoded against `background: var(--color-accent)`, which passes on Darcula's dark red and fails on seven lighter accents (worst 2.00:1 on Nord). Whole catalogue: 171 violation nodes across 16 themes pre-fix, 0 post-fix, 0 newly introduced.
- **Why this is recorded as automated rather than out-of-band (§11.4.6):** the briefing for this audit stated all four defects were out-of-band. For BOB-164's *original* discovery the repository's own filing record contradicts that, and recording an automated discovery as out-of-band would falsify the §11.4.238(E) split in the pessimistic direction — as forbidden as flattering it. The genuinely out-of-band escape inside this item is real and is audited separately as **164b**.
- **Standing-coverage note (§11.4.226), not an escape audit:** the suite that made this discovery, `tests/ux/` (`test_live_dashboard_accessibility.py`, `test_accessibility_axe.py`, `test_keyboard_navigation.py`, with `golden-good/golden-bad` fixtures), **is tracked in git** — but a needed search (`tests/unit` found in `ci.sh` 1×, `scripts/` `pre_build_verification.sh` 10×, `run-all-tests.sh` 1×; `tests/ux` found 0× in all three, and 0 files repo-wide reference it) shows **no runner script executes it**. Its next run depends on someone choosing to invoke it. That is the freshness half of §11.4.226 — registration is not coverage — and wiring `tests/ux/` into a standing runner is owed follow-up work, filed here rather than performed (`ci.sh` and the pre-build script are held by live sibling agents).

BOB-164b — the round-1 contrast fix drove a background-role token from 2.46:1 to 1.09:1, and both oracles were blind to it

- **id:** BOB-164 (round 2) — the regression introduced by the round-1 fix to 164a

- **date:** 2026-08-23 (frontend/src/app/models/style-contrast.spec.ts header; docs/qa/BOB-164/README.md)
- **channel:** agent-code-reading — found by an agent reasoning over the stylesheets while authoring round 2, after both existing oracles had reported clean.
- **summary:** --color-text-secondary is used as a **foreground** in eleven places and as a **background** in at least one:
`.status.unknown { background: var(--color-text-secondary); color: #ccc }.` Round 1 lightened the token to clear the 4.5:1 floor in its foreground role, and in doing so drove its background role **from 2.46:1 to 1.09:1** — a disabled hook's "No" rendering at barely above the 1:1 floor. A safe change to one role is a regression in the other, and nothing in the regime connected the two.
- **escape-audit: Three oracles covered this surface and all three were blind at once**, each for a different, structural reason:
 1. **The new arithmetic oracle was blind by construction.**
 frontend/src/app/models/palette.contrast.spec.ts — authored in round 1, the check that was supposed to close 164a — asserts TEXT tokens against SURFACES = [bgPrimary, bgSecondary, bgTertiary]. A token used as a **background** is outside its pair set **by construction**, so the very fix it was written to validate moved a pair it could not name. This is the §11.4.245 oracle-scope failure: the pair set was drawn from the token's *intended* role, not from the pairs the stylesheets actually declare.
 2. **The rendered-DOM oracle never saw the node.** axe only reports what renders, and the results table and the hooks list **render EMPTY against a static dist with no backend** — so the `.status.unknown` node was not in the DOM axe scanned. Two oracles, two different blindnesses, one uncovered pair.
 3. **The long-standing catalogue test asserts identity, not contrast — and contains the word anyway.** tests/unit/test_palette_catalog.py asserts token *presence* (a REQUIRED_TOKENS tuple), hex *shape* (`^[0-9a-fA-F]{6}$`), and one exact value: `assert darcula["dark"]["accent"].lower() == "#9d001e"`. That is a brand **lock** — it pins the precise hex that measured 1.24:1 as text, and is green whether the palette is readable or not. Its single occurrence of the string contrast is the token **name** "contrast" at line 41 inside REQUIRED_TOKENS — a carrier, verified by reading the line rather than trusting the count of 1. The word was present in the covering test for the entire life of the defect and no ratio was ever computed.
 4. **The one UI-design gate computes no ratio and does not block.** Invariant 41 CM-OPENDESIGN-UI-SYSTEM is **ADVISORY**, and its sub-checks are hardcoded-hex-in-theme-sources, design-

token-artifact presence, and visual-regression-suite presence.
None of them is a contrast assertion.

- **new-check:** `frontend/src/app/models/style-contrast.spec.ts` — and the important property is that it removes the **class**, not the instance. Rather than patching four ratios, it **reads the real stylesheets, extracts every (foreground, background) pair they declare**, and asserts each across all sixteen palette × mode combinations. A token used as a fill is inside the pair set by construction, and any NEW fill anyone adds is picked up without editing the spec. Oracle independence is explicit and satisfies §11.4.245: the oracle is `contrastRatio` recomputed from the published WCAG 2.x formula against the constant 4.5, neither derived from the palette nor from the stylesheets under test — strategy SPECIFIED. The round-1 spec is retained for the token-role layer, and paired §1.1 mutations were run against it, including the one that matters: **M2, replacing an accent with a grey that CLEARS the ratio, is caught by a hue guard** — without it a ratio-only checker accepts turning every brand and error colour grey.
- **Honest boundary (§11.4.6) — this check is authored, not yet enforced, and NOT executed by this audit:**
 1. **Both spec files are UNTRACKED** (`palette.contrast.spec.ts`, `style-contrast.spec.ts`). Per §11.4.215 an untracked artifact does not bind.
 2. **No standing gate runs them.** A needled search for `ng test / vitest / npm test` across `ci.sh`, `scripts/pre_build_verification.sh` and `run-all-tests.sh` returns zero (the same `grep` finds `pytest` 10× in `ci.sh`, so it can see). The only `vitest` reference in the pre-build script is a comment about bounding a wedged invocation.
 3. **This audit did not execute them.** `frontend/**` is held by a live sibling agent, so the tree is not quiescent; a verdict measured on a racing tree is void (§11.4.84) and would be worse than no verdict. The GREEN figures quoted in `docs/qa/B0B-164/README.md` (652 passed; 0 violation nodes across 16 themes) are the item's captured evidence, read for this audit and attributed, **not** re-measured by it.
 4. The item's own evidence records three further gaps that remain open regardless: `onAccent` falls short against **accentHover** in four combinations (3.74–3.97); roughly twelve `color: #fff-on-semantic-fill` sites were exercised by no scan and are neither proven broken nor proven safe; and the compiled bundle in `download-proxy/src/ui/dist/` was deliberately not rebuilt, so §11.4.108 layer 2 is not closed and the served bundle still carries the defect.

5. Contrast is not legibility. No human has looked at these colours;
§11.4.185 manual QA is unaffected and still owed.

BOB-174 — a corrupt hooks file read as zero hooks, and the covering test asserted exactly that

- **id:** BOB-174 (status Queued in docs/workable_items.db)
- **date:** 2026-08-23 (docs/qa/BOB-174/EVIDENCE.md)
- **channel:** agent-code-reading — found by an agent tracing the corrupt-load path in download-proxy/src/api/hooks.py while following on from BOB-173, and confirmed by writing the guard first and running it against the unmodified tree.
- **summary:** `_load_hooks()` swallowed a parse error and returned `[]`, so a truncated or malformed `hooks.json` presented to the API as “**no hooks**”. The next create then wrote a fresh single-element store over it — turning a hand-repairable truncated file into an unrecoverable one. Reproduced verbatim against the unmodified tree: “*prod-hook-0’ is no longer in the hook store. A truncated file is hand-repairable; an overwritten one is not.*” Three operator hooks in, one unrelated hook out. The chain closes on itself: a write that failed part-way left `{“hook_id”: “half-writ on disk — which is precisely the corrupt input the load path then reads as zero and overwrites. delete reported 404 not found for a store it simply could not read. Guard-first RED against the unmodified tree: 15 failed, 13 passed.`
- **escape-audit: The covering test asserted the defect.** This is not a scope gap or a blind instrument — the regime was green *on* the defect and would have gone red on the fix. Verified for this audit against HEAD’s committed blob, not from the evidence pack: `git show HEAD:tests/unit/api_layer/test_hooks_coverage.py, test_load_hooks_invalid_json, lines 54–61:`

```
fake_path.write_text("not json")
hooks = api.hooks._load_hooks()
assert hooks == []           # the defect, codified as the
                             expectation
```

That is the §11.4.245 oracle failure in its purest form: the expected value was derived from what the implementation *did*, never from a specification of what a corrupt store *should* cause. Any future correct fix was pre-committed to breaking this test, which is exactly why the swallow “looked deliberate” and survived. The negative

control that makes the distinction meaningful was there all along and is correctly untouched: `test_load_hooks_no_file` still asserts `== []` for a **missing** file (HEAD lines 25–31; still present in the working tree at line 31) — MISSING and CORRUPT are different states, and only one of them is legitimately empty.

A second, independent mechanism kept the write half quiet: **BOB-173's guard was aimed at an operation the code no longer performs**. `seal_for_rewrite` `chmod`'ed the hooks **file** to 0400 and probed with `open(target, "a")` — the correct needle while `_save_hooks` rewrote in place. Measured rather than assumed: append to a 0400 file is **REFUSED**; `os.replace` over a 0400 file **SUCCEEDS**; `mkstemp` in a 0500 directory is **REFUSED**. So a guard written against in-place rewriting silently stops biting the moment the write becomes atomic — the §11.4.199 shape, a reproduction that no longer reaches its own precondition.

- **new-check:** `tests/unit/api_layer/test_bob174_corrupt_hook_store.py`. Executed for this audit (collection only — see the boundary below): **35 tests collected**, cleanly, which also proves the current `hooks.py` imports. The item's evidence pack header says 28; the suite grew afterwards and the two figures are left un-harmonised (§11.4.6). Its shape is what makes it a class-closing check rather than an instance patch: corrupt-store behaviour is asserted for **three distinct corruption seeds** (`truncated`, `wrong_shape`, `list_of_non_dicts`) across GET, CREATE, DELETE and dispatch; atomicity is proven by a crash mid-write leaving the previous store intact **and still parseable**; and a non-`OSError` serialisation failure is pinned separately. The two gates the fix legitimately broke were **reconciled, not fake-passed** (§11.4.120) — `test_hooks_coverage.py` now expects `pytest.raises(HookStoreCorruptError)`, and the three BOB-173 DELETE tests were re-aimed at the **directory** seal, which is a closer match to the operation the code now performs than the original was. Both reconciliations were then re-verified to still bite: restoring BOB-173's `_save_hooks` swallow → 5 failed; restoring the A1 `_load_hooks` swallow → 2 failed. A reconciled gate that stopped catching its own defect would be §11.4.120's fake-pass in slow motion, and that was checked rather than assumed.

- **Honest boundary (§11.4.6):**

1. **The guard is UNTRACKED** and the `hooks.py` change is uncommitted; the item remains Queued. As with BOB-169, the

check exists in one working copy and binds nothing on a fresh clone (§11.4.215).

2. **This audit ran collection, not the suite.** `download-proxy/**` is held by a live sibling agent, so the tree is not quiescent and a pass/fail verdict measured against it would be void (§11.4.84). The RED (15 failed / 13 passed), the GREEN, and the mutation results quoted above are the item's captured evidence, attributed and not re-measured here.
3. The evidence pack states plainly that three RED members passed pre-fix and do **not** capture the escape by themselves — they pin properties of the new mechanism. That honesty is carried forward rather than smoothed over: the members that captured the defect are named there individually.
4. A deliberate behaviour change ships with the fix: a `hooks.json` `chmod`'ed 0400 no longer refuses a write, because `os.replace` relinks a directory entry and never consults the target's mode. It is accepted, argued, and pinned by its own test rather than left as a docstring claim.

Discovery-channel split (tracked, per §11.4.238(E))

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
2026-08-07 → 2026-08-10 (incremental, this period only)	0	8 (agent-code-reading x4, incidental-discovery x3, automated_background_scan x1)	100%
2026-08-11 → 2026-08-12 (incremental, this period only)	0	1 (operator-report x1)	100%
2026-08-18 (incremental, BOB-069 backfill — 4 new ### entries; the BOB-073 recurrence is an addendum to the	0	4 (agent-code-reading x4: RD2-00/BOB-068, BOB-072, META-11.4.227B, SCRATCH-LOSS)	100%

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
existing BOB-008 heading, not a new ###)			
2026-08-18 (incremental, BOB-069-review fix — 1 new ### entry: the missing CodeGraph 1.5.0 nested-.gitignore escape identified in the independent review of BOB-069)	0	1 (agent-code-reading x1: CODEGRAPH-1.5.0-GITIGNORE)	100%
2026-08-18 (incremental, BOB-116 forced-logout incident (initially referenced as BOB-076 informal label, corrected 2026-08-18) — 1 new ### entry: 2nd occurrence of user@1000 SIGKILL on this project after physical operator return, discovered by operator-report)	0	1 (operator-report x1: FORCED-LOGOUT-2026-08-18-2ND)	100%
2026-08-18 (incremental, §11.4.209 review IMPORTANT-2 remedy — 1 new ### entry: docs/	0	1 (agent-code-reading x1: DB-BLOB-COMMITTED-WITHOUT-DELTA-3520621)	100%

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
workable_items.db binary blob committed without differential evidence, discovered by agent-code- reading)			
2026-08-18 (incremental, BOB-120 forced- logout incident — 1 new ### entry: 3rd occurrence of user@1000 SIGKILL on this project, plus the discovery that the BOB-116/task-77 preventive monitor itself lives inside the killed scope and cannot catch a kill landing before its own next fire)	0	1 (agent-code-reading x1: FORCED- LOGOUT-2026-08-18-3RD)	100%
2026-08-19 (incremental, BOB-124 forced- logout incident #5 — 1 new ### entry: 5th consecutive occurrence of user@1000 SIGKILL, escalating the coverage escape	0	1 (operator-report x1: FORCED- LOGOUT-2026-08-19-5TH)	100%

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
from “no preventive gate” to “4 authored preventive gates, 0 installed” — §11.4.250 heuristic-tower / architectural install-gap)			
2026-08-20 (incremental, tooling-defect sweep during a commit/push round — 2 new ### entries: the compute-badges.sh carrier-match README corruption + non-idempotent TESTING.md regeneration, and the repo-wide gitignore-shadows-tracked-files break of the §11.4.234 commit mechanism)			
2026-08-20 (incremental, systematic-debugging of the 3 quarantined bash suites — 1 new ### entry: the non-hermetic dead-scratchpad-path + plain-cp	0	2 (agent-code-reading x2: COMPUTE-BADGES-CARRIER-MATCH, GITIGNORE-SHADOWS-TRACKED-FILES)	100%
	0	1 (agent-code-reading x1: NON-HERMETIC-TESTS-MUTATE-REAL-TREE)	100%

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
mtime-bump root cause behind all three)			
2026-08-20 (incremental, subagent fan-out — 1 new ### entry: invariant 16 mtime staleness is non-reproducible across fresh clones, found while investigating the content-hash migration)	0	1 (agent-code-reading x1: EXPORT-SYNC-MTIME-NOT-REPRODUCIBLE-ON-FRESH-CLONE)	100%
2026-08-21 (incremental, feature 002-user-owned-downloads — 1 new ### entry: every container-produced file owned by uid 100999, reported by the operator while using the product; no automated check had ever asserted a produced file's owner)	0	1 (operator-report x1: BOB-CONTAINER-WRITES-NOT-OPERATOR-OWNED)	100%
2026-08-25 (incremental, coverage-escape audit of four defects found	1 (automated-helixqa x1: BOB-164a, discovered	4 (agent-code-reading x4: BOB-169, BOB-168, BOB-164b, BOB-174)	80%

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
during the 002-user-owned-downloads session — 5 new ### entries: BOB-169 export charset corruption, BOB-168 guard residue, BOB-164a/ BOB-164b split, BOB-174 corrupt hooks store)	by the tests/ux/axe suite on its first live run)		
Cumulative total (all ### entries to date, this row is what CM-QA- DISCOVERY-LEDGER-FRESH checks)	1	27	96%

Pre-existing check-vs-table mismatch, found and fixed during this backfill (§11.4.6, not silently patched around): scripts/pre_build_verification.sh invariant 19 (CM-QA-DISCOVERY-LEDGER-FRESH) computes its expected count from ONLY the LAST row of this table, comparing it against the TOTAL ### heading count under ## Entries. The three “incremental, this period only” rows above were never designed to satisfy that arithmetic on their own (each was authored to show only that update’s delta, not a running total) — re-running the exact invariant-19 awk logic against the pre-backfill file (git show HEAD:docs/QA_DISCOVERY_LEDGER.md) confirms this was **already FAILing before this session’s edits** (entries=9 split-table-declared=1, live-run of scripts/pre_build_verification.sh confirms FAIL [5]). The **Cumulative total** row above is added so the check’s actual last-row-only arithmetic now passes (0+13=13 == the real ### count below); the three incremental rows are kept as honest per-period trend history. This finding is itself a coverage escape of the same shape this ledger tracks (an existed-but-missed gate whose own arithmetic

silently drifted from the document it gates) but is fixed in the same edit that found it since the fix is a one-line table-convention correction, not a new automated check.

Honest note (updated 2026-08-25): the cumulative figure is now **96% out-of-band — 27 of 28 entries** — down from 100%, and the single point of movement is worth stating precisely rather than celebrated. It is **one** entry: BOB-164a, where the tests/ux/ axe suite found a real user-visible defect on its first live run and the failing test was deliberately left failing rather than silenced. That is the §11.4.238 posture working exactly once. Every other entry in this ledger, including the four audited on 2026-08-25, was still found by an agent reading code, running a command by hand, or hitting a real failure during unrelated work.

Direction of travel, stated without editorialising: 100% → 96% over the ledger's life is **one automated discovery in 28**, and the two arithmetic caveats both cut against reading it as a trend. The denominator grew by 5 in this round, so the percentage would have fallen slightly even if the automated column had stayed at 0; and BOB-164a's own follow-on regression (BOB-164b) was itself an out-of-band escape, so the one automated discovery did **not** carry its own item to closure without the regime being escaped again. §11.4.238(E)'s target is this percentage trending toward zero, and on the evidence of this round it has barely begun to.

A structural obstacle is now visible across the round and is recorded here rather than in any single entry, because it is the same finding four times: **every new or strengthened check produced by these four audits is UNTRACKED in git**, and invariant 50's own gate is present only in an uncommitted working tree (HEAD's scripts/pre_build_verification.sh reads [49/49] and contains zero occurrences of CM-EXPORT-CHARSET-VALID). Per §11.4.215 an untracked artifact does not bind, and per §11.4.226 a check that exists in one working copy is not coverage. Until these are committed, the new-check column for this round describes work done, not a regime strengthened — the same install-gap class this ledger already recorded at FORCED-LOGOUT-2026-08-19-5TH ("4 authored preventive gates, 0 installed"). The earlier open items are unchanged: RD2-42/RD2-43 still have no automated check authored, and BOB-168's is owed rather than written.

2026-08-18 backfill note (BOB-069): this round added four entries discovered during the SAME session that produced them (the constitution-curriculum amendment stream + the BOB-072/073/075 parallel fixes), plus one recurrence addendum to the pre-existing

BOB-008 entry. All five remain agent-code-reading — none crossed a standing automated gate. BOB-072 is the closest this ledger has come to the ideal shape: its closing evidence was captured via the sanctioned tool (workable-items validate/diff) rather than raw inspection, even though the discovery itself predates any standing gate invoking that tool automatically. Four followup workable items were filed (BOB-104..BOB-107) for the specific new/strengthened automated checks each entry's new-check field names as not-yet-authored; none of the four checks are claimed as shipped by this backfill (§11.4.6) — they are tracked, not implemented, here.

2026-08-19 BOB-124/125/126 escape — SEVEN forced-logout incidents traced to ONE untested test-defect (chain closed):

Between 2026-08-18 20:50 and 2026-08-19 16:43, **SEVEN** forced-logout incidents (BOB-116 / BOB-120 / BOB-123 / BOB-124 / BOB-125 / BOB-126 — the seventh caught by kernel audit rules installed 15:56 during the sixth investigation) wiped the operator's entire graphical session, tmux server, ssh connections, browsers, and the working Claude Code process — every ~30-40 minutes for ~48 hours.

Actual root cause (§11.4.6-attributed 2026-08-19 16:43:43 via audit[399861]: SYSCALL syscall=62 a0=ffffffff a1=9 pid=399861 comm=\"pytest\" exe=\"/usr/bin/python3.14\"): tests/unit/merge_service/test_deadline_tunable.py::test_deadline_hit_flag_true_when_readline_times_out created AsyncMock() without setting mock.pid as an int. Production_search_public_tracker code path called os.killpg(os.getpgid(proc.pid), signal.SIGKILL). Python's MagicMock.__int__ defaults to 1, so os.getpgid(int(MagicMock())) == 1 → os.killpg(1, SIGKILL) → glibc → kill(-1, SIGKILL) = **SIGKILL every UID-1000 process**. Bug existed since 2026-04-24 (~4 months). contextlib.suppress(Exception) swallowed nothing because the syscall SUCCEEDED before any exception.

Discovery channel: operator-report (7 times). **Coverage-escape audit** (§11.4.238(C)): the automated regime had NO check for “does a production-code function ever call killpg with pgid ≤ 1?” or “does a test create subprocess mocks without explicit int pid?”. §11.4.115 RED-first was violated: the test that TRIGGERED the disaster had no assertion that the code path avoided the disaster. New check landed: tests/unit/merge_service/test_deadline_tunable.py::test_bob126_regression_deadline_path_never_calls_killpg_with_pgid_le_1 (see commit ad4b46a) — a §11.4.115 RED-first regression guard asserting os.killpg is NEVER called with

$\text{pgid} \leq 1$ even under the exact defect precondition. Reverts of the production-code pid-guard MUST make this test fail. Also NEW universal anchor **§11.4.263** — process-group signal-safety mandate (Constitution commit 502586c) — extends the mandate to every project inheriting the constitution (Python/Go/Rust/Bash/C) with four-layer coverage per §11.4.4(b).

§11.4.238 discipline preservation (agent-code-reading DID FIND, sort of — but only on the 7th time, and only via kernel audit rules the operator ran a `su -c` command to install manually on the same day; the prior 6 investigations misattributed the mechanism to PAM/Linger contradictions and other downstream effects — all wrong per §11.4.6, honest self-correction at Rev 4 of the incident docs). This is the ledger's LARGEST out-of-band escape by user-visible impact.

- **BOB-124** (5th incident): status=Fixed via BOB-126 root-cause. Incident doc filed 2026-08-19 15:35, corrected retrospectively.
- **BOB-125** (6th): status=Fixed via BOB-126. Doc filed 16:11, Rev 4 correction 16:15.
- **BOB-126** (7th): status=Fixed. Doc filed + closed same day.
- Prior 4 (BOB-116, BOB-120, BOB-123 + the base BOB-116's "first" instance): retrospective correction pending — their attribution to PAM/Linger was wrong per BOB-126's captured evidence.

2026-08-25 export-staleness note (§11.4.65 / §11.4.12, owed not performed): this revision edits docs/QA_DISCOVERY_LEDGER.md only. Its .html, .pdf and .docx siblings are therefore **STALE** — measured, not assumed, with the project's own oracle (`scripts/lib/export_staleness.sh export_is_stale`), which reports STALE for both .html and .pdf. They were deliberately **not** regenerated: the working tree is not quiescent (three sibling agents hold `scripts/`, `download-proxy/**` and `frontend/**` during this audit), and running the corpus generator against a racing tree is how a half-written export gets committed (§11.4.84, §11.4.121). Consequence stated plainly so it is not a surprise: **invariant 16 CM-MARKDOWN-EXPORT-SYNC will FAIL on these two siblings until they are regenerated**, and the regeneration must run HTML-before-PDF or weasyprint rebakes the mojibake from the stale fragment (BOB-169). Related and recorded in the BOB-169 entry above: this ledger's own .html is currently one of the 301 charset-less fragments and its .pdf carries 494 corrupted lines, so the regeneration is also the repair.